

Media files

Media files are uploaded by users and should never be trusted. So, for example, don't develop a Django app that interprets uploaded Python files.

Consider what your backup strategy is for uploaded content.

HTTPS

Data leakage over insecure HTTP connections is a perennial issue in web security. These two settings force the use of HTTPS connections to transmit sensitive data.

CSRF_COOKIE_SECURE

Setting this variable to `True` enforces the transmission of the CSRF cookie over HTTPS. We typically need this set to `False` while we are developing on `http://localhost:8000` and don't have a secure connection available, but it is imperative to set it to `True` in production so that the CSRF cookie is not transmitted over an insecure protocol.

SESSION_COOKIE_SECURE

Similarly, `SESSION_COOKIE_SECURE=True` enforces that the session cookie is transmitted over HTTPS.

Performance optimizations

These are some of the performance optimizations that should be considered prior to deployment.

Sessions

It is recommended that a cached session package be used, as it has the advantage of reducing the impact on the database. Also, when we are developing our application, we are generating a low rate of user interaction, but in a production environment, there is significant session activity, and the database tables that maintain this can grow incrementally if older session data is not periodically purged.

CONN_MAX_AGE

The `CONN_MAX_AGE` setting allows database connections to persist on a web server thread between requests. This means that if a worker thread is busy sequentially servicing requests, it does not need to make separate database connections for each, as a persistent connection has been established.

미디어 파일

미디어 파일은 사용자에 의해 업로드된 idx_52093512이며 절대 신뢰해서는 안 됩니다. 예를 들어, 업로드된 Python 파일을 해석하는 Django 앱을 개발하지 마십시오.

업로드된 콘텐츠에 대한 백업 전략이 무엇인지 고려하십시오.

HTTPS

불안정한 HTTP 연결을 통한 데이터 유출 idx_77e540a5는 웹 보안에서 지속적인 문제입니다. 이 idx_7ba412ab두 설정은 민감한 데이터를 전송하기 위해 HTTPS 연결을 사용하도록 강제합니다.

CSRF_COOKIE_SECURE

이 idx_1f1746a7변수를 True로 설정하면 CSRF 쿠키가 HTTPS를 통해 전송되도록 강제됩니다. 우리는 일반적으로 http://localhost:8000에서 개발하는 동안 이 값을 False로 설정해야 하며, 안전한 연결이 없지만, 프로덕션에서는 CSRF 쿠키가 불안정한 프로토콜을 통해 전송되지 않도록 True로 설정하는 것이 필수적입니다.

SESSION_COOKIE_SECURE

마찬가지로, SESSION_COOKIE_SECURE=True는 세션 쿠키가 HTTPS를 통해 전송되도록 강제합니다.

성능 최적화

이것들은 배포 전에 고려해야 할 성능 최적화의 일부입니다.

세션

캐시된 세션 패키지를 사용하는 것이 권장됩니다. 이는 데이터베이스에 미치는 영향을 줄이는 장점이 있습니다. 또한, 애플리케이션을 개발할 때는 사용자 상호작용이 낮은 비율로 발생하지만, 프로덕션 환경에서는 상당한 세션 활동이 있으며, 이를 유지하는 데이터베이스 테이블은 이전 세션 데이터가 주기적으로 삭제되지 않으면 점진적으로 증가할 수 있습니다.

CONN_MAX_AGE

CONN_MAX_AGE 설정은 데이터베이스 연결이 요청 간에 웹 서버 스레드에서 지속되도록 허용합니다. 이는 작업 스레드가 요청을 순차적으로 처리하는 동안 각 요청에 대해 별도의 데이터베이스 연결을 만들 필요가 없음을 의미합니다. 이미 지속적인 연결이 설정되어 있습니다.

TEMPLATES

Template caching improves Django performance as it means that templates do not need to be read from disk and compiled into Python for each request, but remain in memory as Python template objects. The default behavior since Django 4.1 is to enable template caching, and it is implemented internally in Django, independently of caching mechanisms, and doesn't require the explicit configuration of a cache (as is the case with session or view caching).

This means that the default template loader is set to `django.template.loaders.cached.Loader` unless it has been explicitly disabled for debugging purposes. If you have disabled template loading in the development environment, be sure to re-enable it during deployment.

Error reporting

LOGGING

During debugging, we typically view logging in the console and traceback errors in our application browser. A production environment should produce logging that is more persistent and trackable.

ADMINS and MANAGERS

The default behaviour of Django's error logging is to send an email to everyone in the `ADMINS` list if there is a 500 error. `ADMINS` is a list of email addresses configured in the `<project>/settings.py` file. By default, it is empty, but it is good to add relevant email addresses of developers and administrators in order to receive information about 500 errors, since this information is no longer displayed in the browser when `DEBUG` is `False`.

Similarly, the `MANAGERS` list is configured in the `<project>/settings.py` file. Recipients in this list receive emails about broken links in the project when the `BrokenLinkEmailsMiddleware` is enabled:

```
ADMINS = ['systemadmin@bookr.example.com',
          'alice@example.com']
MANAGERS = ['contentmanager@bookr.example.com',
            'david@example.com', 'eve@example.com']
```

Customize the default error views

Django ships with default templates for 400, 403, 404, and 500 errors. It might be preferable to render these errors in a manner that is consistent with the rest of your project's design. This can be done by adding your own `400.html`, `403.html`, `404.html`, and `500.html` template files into the project's root template folder.

템플릿

템플릿 캐싱 `idx_44d0f2f7`은 장고 성능을 향상시킵니다. 이는 템플릿이 디스크에서 읽고 각 요청에 대해 파이썬으로 컴파일될 필요가 없으며, 파이썬 템플릿 객체로 메모리에 남아있음을 의미합니다. 장고 4.1 이후의 기본 동작은 템플릿 캐싱을 활성화하는 것이며, 이는 장고 내부에서 구현되어 캐싱 메커니즘과 독립적으로 작동하며, 캐시(세션 또는 뷰 캐싱의 경우와 같이)를 명시적으로 구성할 필요가 없습니다.

이는 기본 템플릿 로더가 `django.template.loaders.cached.Loader`로 설정되어 있음을 의미하며, 디버깅 목적으로 명시적으로 비활성화되지 않은 경우에 해당합니다. 개발 환경에서 템플릿 로딩을 비활성화한 경우, 배포 중에 다시 활성화해야 합니다.

오류 보고

LOGGING

디버깅 중 `idx_2f258135`, 우리는 일반적으로 콘솔에서 로그를 보고 애플리케이션 브라우저에서 오류 추적을 확인합니다. 프로덕션 환경에서는 더 지속적이고 추적 가능한 로그를 생성해야 합니다.

ADMINS 및 MANAGERS

장고의 오류 로깅의 기본 동작은 500 오류가 발생할 경우 ADMINS 목록에 있는 모든 사람에게 이메일을 보내는 것입니다. ADMINS는 `<project>/settings.py` 파일에 구성된 이메일 주소 목록입니다. 기본적으로 비어 있지만, 500 오류에 대한 정보를 받기 위해 개발자 및 관리자와 관련된 이메일 주소를 추가하는 것이 좋습니다. 이 정보는 DEBUG가 False일 때 브라우저에 더 이상 표시되지 않습니다.

유사하게, MANAGERS 목록 `idx_f056bb3f`은 `<project>/settings.py` 파일에 구성되어 있습니다. 이 목록의 수신자는 BrokenLinkEmailsMiddleware가 활성화된 경우 프로젝트의 끊어진 링크에 대한 이메일을 받습니다:

```
ADMINS = ['systemadmin@bookr.example.com',
          'alice@example.com']

MANAGERS = ['contentmanager@bookr.example.com',
            'david@example.com', 'eve@example.com']
```

기본 오류 뷰 사용자 정의

Django는 400, 403, 404 및 500 오류에 대한 기본 템플릿을 제공합니다. 이러한 오류를 프로젝트의 나머지 디자인과 일관되게 렌더링하는 것이 바람직할 수 있습니다. 이는 프로젝트의 루트 템플릿 폴더에 자신의 400.html, 403.html, 404.html 및 500.html 템플릿 파일을 추가하여 수행할 수 있습니다.

With these considerations in mind, we can set about configuring our Bookr project for deployment in the following exercise.

Exercise 15.01 – Configuring the Bookr project for deployment

In this exercise, we are challenged with applying the Django deployment checklist to our Bookr project before it is ready for a production environment:

1. Our first concern with the Bookr project is that the `bookr/settings.py` file contains the value of the `SECRET_PASSWORD` setting. A safer option is to load it from either an environment variable or an external file, so that it is decoupled from the source code of the Django project. We will also add a variable in our settings that acts as a switch. It will be called `ENVIRONMENT`, and its value will either be `'Development'` or `'Production'`.

In *Chapter 10*, we introduced the `decouple` module and imported it into `bookr/settings.py`. We started a `.env` file to include the sensitive or environment-specific settings:

```
from decouple import config
```

```
SECRET_KEY = config('SECRET_KEY')
ENVIRONMENT = config('ENVIRONMENT')
```

We will need to add the `SECRET_KEY` and `ENVIRONMENT` settings to the `.env` file that we created in the top-level directory of our project:

```
SECRET_KEY=...
ENVIRONMENT=development
```

While the existing `.env` file is being used in our development environment, we can create another `.env` file in a secure location where we will store values that we set for production. In this exercise, we will refer to these files as `development .env` and `production .env` in order to avoid confusion.

We can use this command to generate a strong value for the secret key in the `production .env`, before adding it to the file:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

이러한 고려 사항을 염두에 두고, 다음 연습을 위해 Bookr 프로젝트를 배포하도록 구성할 수 있습니다.

연습 15.01 – Bookr 프로젝트 배포 구성

이 idx_032f8892 연습에서는 Bookr 프로젝트가 프로덕션 환경에 준비되기 전에 Django 배포 체크리스트를 적용하는 도전 과제가 주어집니다:

1. Bookr 프로젝트에 대한 첫 번째 우려 사항은 bookr/settings.py 파일에 SECRET_PASSWORD 설정의 값이 포함되어 있다는 것입니다. 더 안전한 옵션은 환경 변수나 외부 파일에서 로드하여 Django 프로젝트의 소스 코드와 분리하는 것입니다. 또한 설정에 스위치 역할을 하는 변수를 추가할 것입니다. 이 변수는 ENVIRONMENT라고 불리며, 값은 'Development' 또는 'Production'이 될 것입니다.

10장에서 decouple 모듈을 소개하고 bookr/settings.py에 가져왔습니다. 민감한 또는 환경별 설정을 포함하기 위해 .env 파일을 시작했습니다:

```
from decouple import config
```

```
SECRET_KEY = config('SECRET_KEY')
ENVIRONMENT = config('ENVIRONMENT')
```

우리는 프로젝트의 최상위 디렉토리에 생성한 .env 파일에 SECRET_KEY와 ENVIRONMENT 설정을 추가해야 합니다:

```
SECRET_KEY=...
ENVIRONMENT=development
```

기존 .env 파일이 개발 환경에서 사용되고 있는 동안, 우리는 프로덕션을 위해 설정한 값을 저장할 안전한 위치에 또 다른 .env 파일을 생성할 수 있습니다. 이 연습에서는 혼동을 피하기 위해 이러한 파일을 개발 .env와 프로덕션 .env라고 부를 것입니다.

이 명령을 사용하여 프로덕션 .env에 비밀 키에 대한 강력한 값을 생성할 수 있습니다. 파일에 추가하기 전에:

```
python -c "from django.core.management.utils import get_random_secret_key;
print(get_random_secret_key())"
```

```
SECRET_KEY=...  
ENVIRONMENT=production
```

2. Using the `ENVIRONMENT` switch, we can ensure that `DEBUG` is only set to `True` in development environments:

```
DEBUG = True if config(ENVIRONMENT)=='development' \  
        else False
```

3. At present, our project has no allowed hosts set. We can use the `config` function to load the value from the `.env` file. For now, we will just add a wildcard value for `ALLOWED_HOSTS` in production `.env`, but if we had a domain name for our production server, this is where we would add it.

By default, `decouple.config` loads values from the `.env` file as strings. As `ALLOWED_HOSTS` is a list, we need a method of casting it. We can use the `Csv()` method of `decouple` that parses a setting using the comma-separated value format that we are familiar with. So, if the `.env` file contains `ALLOWED_HOSTS=a.site,b.site`, then it will be parsed by `config` as `['a.site', 'b.site']`:

```
from decouple import config, Csv  
  
ALLOWED_HOSTS = config('ALLOWED_HOSTS', Csv())
```

The development `.env` file will contain `ALLOWED_HOSTS=`, which will be parsed as an empty list.

We will provisionally set the wildcard in the production `.env` file by adding `ALLOWED_HOSTS=*`.

4. The current database configuration points to a SQLite3 database in the project folder, `sqlite3.db`. We will keep this for development, but add a PostgreSQL configuration in our production environment. At present, the settings that we will use here are placeholders, but we will change them later during deployment. In order to connect to a PostgreSQL database, we need to install the `psycopg-binary` library:

```
pip install psycopg2-binary
```

```
SECRET_KEY=...
환경=생산
```

2. ENVIRONMENT 스위치를 사용하여 DEBUG가 True로 설정되는 것을 보장할 수 있습니다.

개발 환경:

```
DEBUG = True if config(ENVIRONMENT)=='development' \
    아니면 거짓
```

3. 현재 우리 프로젝트에는 허용된 호스트가 설정되어 있지 않습니다. config 함수를 사용하여 .env 파일에서 값을 로드할 수 있습니다. 지금은 production.env에 ALLOWED_HOSTS에 대한 와일드카드 값을 추가할 것이지만, 만약 우리가 production 서버에 대한 도메인 이름이 있다면, 여기에서 추가할 것입니다. 기본적으로 decouple.config는 .env 파일에서 값을 문자열로 로드합니다. ALLOWED_HOSTS는 목록이므로 이를 캐스팅할 방법이 필요합니다. 우리는 익숙한 심표로 구분된 값 형식을 사용하여 설정을 구문 분석하는 decouple의 Csv() 메서드를 사용할 수 있습니다. 따라서 .env 파일에 ALLOWED_HOSTS=a.site,b.site가 포함되어 있다면, config에 의해 ['a.site', 'b.site']로 구문 분석됩니다:

```
from decouple import config, Csv

ALLOWED_HOSTS = config('ALLOWED_HOSTS', Csv())
```

개발 .env 파일에는 ALLOWED_HOSTS=가 포함되어 있으며, 이는 빈 목록으로 구문 분석됩니다.

production.env 파일에 ALLOWED_HOSTS=*를 추가하여 임시로 와일드카드를 설정할 것입니다.

4. 현재 데이터베이스 구성은 프로젝트 폴더의 SQLite3 데이터베이스인 sqlite3.db를 가리킵니다. 우리는 개발을 위해 이를 유지하겠지만, production 환경에 PostgreSQL 구성을 추가할 것입니다. 현재 여기에서 사용할 설정은 자리 표시자이며, 배포 중에 나중에 변경할 것입니다. PostgreSQL 데이터베이스에 연결하기 위해 psycopg2-binary 라이브러리를 설치해야 합니다:

```
pip install psycopg2-binary
```


The standard way of configuring a Django database connector in the `settings.py` file is to specify a default database that contains keys for database `ENGINE`, `NAME`, `USER`, `PASSWORD`, `HOST`, and `PORT`. Configuring six different parameters for a database is an error-prone approach, particularly if the data is being split between the `settings.py` and `.env` files, and it would be preferable to specify a single database connection URL. Although Django does not have this functionality out of the box, there is a Python module that implements it - `dj-database-url`.

We can install it using `pip`:

```
pip install dj-database-url
```

Once installed, we can specify a database string by using the `parse` function:

```
dj_database_url.parse(  
    'postgres://user:password@host:5432/database',  
    conn_max_age=600,  
    conn_health_checks=True)
```

Our intention is to use PostgreSQL in production, but we would like to have the option to run it in our development environment too, so we don't want to use the `ENVIRONMENT` variable to switch between the two.

A better strategy is to check whether the database URL is defined in the `.env` file and to follow that if it is present, or fall back to a SQLite3 database if it is absent.

We will also configure settings for the `conn_max_age` and `conn_health_checks` parameters of the `dj_database.parse` function:

bookr/settings.py

```
DATABASE_URL = config('DATABASE_URL', default='')  
CONN_MAX_AGE = config('CONN_MAX_AGE',  
                      default=0, cast=int)  
CONN_HEALTH_CHECKS = config('CONN_HEALTH_CHECKS',  
                             default=False, cast=bool)  
  
if DATABASE_URL:  
    DATABASES = {  
        'default': dj_database_url.parse(  
            DATABASE_URL,
```

Django 데이터베이스 커넥터를 settings.py 파일에서 구성하는 표준 `idx_bc288982` 방법은 데이터베이스 ENGINE, NAME, USER, PASSWORD, HOST 및 PORT에 대한 키를 포함하는 기본 데이터베이스를 지정하는 것입니다. 데이터베이스에 대해 여섯 가지 다른 매개변수를 구성하는 것은 오류가 발생하기 쉬운 접근 방식이며, 특히 데이터가 settings.py와 .env 파일 간에 분할되는 경우에는 단일 데이터베이스 연결 URL을 지정하는 것이 바람직합니다. Django는 기본적으로 이 기능을 제공하지 않지만, 이를 구현하는 Python 모듈이 있습니다 - `dj-database-url`.

우리는 pip를 사용하여 설치할 수 있습니다:

```
pip install dj-database-url
```

설치가 완료되면 `parse` 함수를 사용하여 데이터베이스 문자열을 지정할 수 있습니다:

```
dj_database_url.parse(
    'postgres://user:password@host:5432/database',
    conn_max_age=600,
    conn_health_checks=True)
```

우리의 의도는 프로덕션에서 PostgreSQL을 사용하는 것이지만, 개발 환경에서도 실행할 수 있는 옵션을 원하므로 두 가지를 전환하기 위해 ENVIRONMENT 변수를 사용하고 싶지 않습니다.

더 나은 전략은 데이터베이스 URL이 .env 파일에 정의되어 있는지 확인하고, 존재하는 경우 그것을 따르거나, 없으면 SQLite3 데이터베이스로 되돌아가는 것입니다.

우리는 또한 `dj_database.parse` 함수의 `conn_max_age` 및 `conn_health_checks` 매개변수에 대한 설정을 구성할 것입니다:

bookr/settings.py

```
DATABASE_URL = config('DATABASE_URL', default='')
CONN_MAX_AGE = config('CONN_MAX_AGE',
                      default=0, cast=int)
CONN_HEALTH_CHECKS = config('CONN_HEALTH_CHECKS',
                             default=False, cast=bool)

if DATABASE_URL:
    DATABASES = {
        'default': dj_database_url.parse(
            DATABASE_URL,
```

```

        conn_max_age=CONN_MAX_AGE,
        conn_health_checks=CONN_HEALTH_CHECKS)
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }

```

So, if we don't intend to use a database for the development environment, we just don't include these settings in development .env, and the project will be configured with a SQLite3 database.

If we do have a PostgreSQL database, we might include something similar to what follows in the .env file (we will obtain the actual connection parameters during our Render .com deployment in *Exercise 15.02 – Deploying the Bookr project*):

.env

```

DATABASE_URL=postgres://<user>:<password>@<host>:<port>/<database>
CONN_MAX_AGE=600
CONN_HEALTH_CHECKS=True

```

5. Our development server serves the files from STATIC_DIR. However, in our production environment, this functionality is decoupled from the gunicorn server and either requires a standalone server such as NGINX to serve the static files or the use of middleware such as Whitenoise, which co-opts gunicorn for the purpose. For ease of deployment, we will install Whitenoise:

```
pip install whitenoise
```

We will also need to specify this middleware in the settings.py file and assign a location for STATIC_ROOT where the collectstatic process can save the files during deployment:

bookr/settings.py

```

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',

```

```

        conn_max_age=CONN_MAX_AGE,
        conn_health_checks=CONN_HEALTH_CHECKS)
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }
}

```

따라서 개발 환경에서 데이터베이스를 사용할 의도가 없다면, 이러한 설정을 `development.env`에 포함하지 않으면 되고, 프로젝트는 SQLite3 데이터베이스로 구성됩니다.

만약 PostgreSQL 데이터베이스가 있다면, `.env` 파일에 다음과 유사한 내용을 포함할 수 있습니다 (실제 연결 매개변수는 Exercise 15.02 – Bookr 프로젝트 배포 중 Render.com 배포 시 얻을 것입니다):

.env

```

DATABASE_URL=postgres://<user>:<password>@<host>:<port>/<database>
CONN_MAX_AGE=600
CONN_HEALTH_CHECKS=True

```

5. 우리의 개발 서버는 `STATIC_DIR`에서 파일을 제공합니다. 그러나 우리의 프로덕션 환경에서는 이 기능이 gunicorn 서버와 분리되어 있으며, 정적 파일을 제공하기 위해 NGINX와 같은 독립 서버가 필요하거나 Whitenoise와 같은 미들웨어를 사용해야 합니다. 배포의 용이성을 위해 Whitenoise를 설치하겠습니다:

```
pip install whitenoise
```

`settings.py` 파일에서 이 미들웨어를 지정하고 `collectstatic` 프로세스가 배포 중에 파일을 저장할 `STATIC_ROOT`의 위치를 지정해야 합니다:

bookr/settings.py

```

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',

```

```
        'whitenoise.middleware.WhiteNoiseMiddleware',
        ...
    ]
    ...
    STATIC_ROOT = BASE_DIR / 'staticfiles'
```

While we won't make any changes to `MEDIA_ROOT` and `MEDIA_URL` in this setup, it is worth noting that not all hosted deployment environments use persistent file storage environments.

6. In the HTTPS settings, we want to ensure that CSRF cookies and sessions are only ever served over HTTPS in production. As our development environment uses HTTP – `http://localhost:8000/` – we don't want to force this behavior in a development environment: **`bookr/setting.py`**

```
if config(ENVIRONMENT)=='development':
    CSRF_COOKIE_SECURE = False
    SESSION_COOKIE_SECURE = False
else:
    CSRF_COOKIE_SECURE = True
    SESSION_COOKIE_SECURE = True
```

These are the essential changes that need to be made to the `bookr/settings.py` and `.env` files in order to prepare for deployment.

You can find the complete code in the Chapter15 folder in this book's GitHub repo.

Deployment options for Django

- While we have settled on a high-performance database and web server, such as gunicorn, we are still presented with several options for deployment of the Django project. There is a range of solutions that are listed here.

Manual deployment

- Traditionally, Django projects were manually deployed by system administrators with a knowledge of the Linux command-line environment. Databases and web servers were manually configured and maintained by these administrators. This kind of approach might be appealing if you have existing infrastructure to deploy to, but for many developers today, it requires additional resources and skill sets outside of their abilities.

```
'whitenoise.middleware.WhiteNoiseMiddleware',
...

STATIC_ROOT = BASE_DIR / 'staticfiles'
```

우리는 이 설정에서 MEDIA_ROOT와 MEDIA_URL에 대한 변경을 하지 않을 것이지만, 모든 호스팅 배포 환경이 지속적인 파일 저장 환경을 사용하지는 않는다는 점은 주목할 가치가 있습니다.

6. HTTPS 설정에서는 CSRF 쿠키와 세션이 프로덕션에서 HTTPS를 통해서만 제공되도록 해야 합니다. 우리의 개발 환경은 HTTP -`http://localhost:8000/` -를 사용하므로, 개발 환경에서 이 동작을 강제하고 싶지 않습니다:

bookr/setting.py

```
if config(ENVIRONMENT)=='development':
    CSRF_COOKIE_SECURE = False
    SESSION_COOKIE_SECURE = False
else:
    CSRF_COOKIE_SECURE = True
    SESSION_COOKIE_SECURE = True
```

배포를 준비하기 위해 bookr/settings.py 및 .env 파일에 필요한 필수 변경 사항입니다.

이 책의 GitHub 저장소의 Chapter15 폴더에서 전체 코드를 찾을 수 있습니다.

Django의 배포 옵션

- 우리는 gunicorn과 같은 고성능 데이터베이스 및 웹 서버에 정착했지만, 여전히 Django 프로젝트의 배포를 위한 여러 옵션이 있습니다. 여기에는 다양한 솔루션이 나열되어 있습니다.

수동 배포

- 전통적으로, Django idx_2d72ee84projects는 Linux idx_3910c574명령줄 환경에 대한 지식을 가진 시스템 관리자가 수동으로 배포했습니다. 데이터베이스와 웹 서버는 이러한 관리자가 수동으로 구성하고 유지 관리했습니다. 이러한 접근 방식은 배포할 기존 인프라가 있는 경우 매력적일 수 있지만, 오늘날 많은 개발자에게는 그들의 능력을 넘어서는 추가 리소스와 기술 세트를 요구합니다.

Containerized deployment

Containerized deployment involves developing "containers" for the configuration of lightweight virtual server modules that specify server requirements and application code necessary for the deployment using pre-existing platform modules.

Docker is the entry-level containerization platform, and it interfaces with an ecosystem of high-performance tools such as Docker Compose, Docker Swarm, and Kubernetes.

This is an example of a Docker file specification needed to deploy and run a minimal Django project:

```
# Specify a pre-built docker container
FROM python:3.12

# Set the working directory in the container
WORKDIR /usr/src/app

# Copy project code into the container's working directory
COPY . .

# Install the dependencies from the requirements file
RUN pip install --no-cache-dir -r requirements.txt

# Expose port 8000 to the host machine
EXPOSE 8000

# Command to run the gunicorn production server
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "bookr:wsgi"]
```

Platform-as-a-service

Platform-as-a-service (PaaS) providers offer a hosting environment that can be utilized without having a background in system administration. These are cloud platforms that will host a Python environment capable of running a Django project and its auxiliary services, such as a database server, backups, and static file hosting.

Popular entry-level PaaS providers include **Render.com**, **PythonAnywhere.com**, **Seenode.com**, and **Railway.com**. The established cloud providers, including Digital Ocean, AWS, and Google Cloud Platform, also cater to PaaS configurations.

컨테이너화된 배포

컨테이너화된 배포는 경량 idx_c3c7cb4f 가상 서버 모듈 idx_547a034a의 구성에 대한 "컨테이너"를 개발하는 것을 포함하며, 이는 배포에 필요한 서버 요구 사항과 애플리케이션 코드를 지정합니다. 기존 플랫폼 모듈을 사용합니다.

도커는 입문 수준의 컨테이너화 플랫폼이며, Docker Compose, Docker Swarm 및 Kubernetes와 같은 고성능 도구의 생태계와 인터페이스합니다.

다음은 최소한의 장고 프로젝트를 배포하고 실행하는 데 필요한 도커 파일 사양의 예입니다:

```
# 미리 빌드된 도커 컨테이너를 지정합니다.
FROM python:3.12

# 컨테이너 내 작업 디렉토리 설정
WORKDIR /usr/src/app

# 컨테이너의 작업 디렉토리에 프로젝트 코드를 복사합니다
COPY . .

# 요구 사항 파일에서 종속성을 설치합니다
RUN pip install --no-cache-dir -r requirements.txt

# 호스트 머신에 포트 8000을 노출합니다
EXPOSE 8000

# gunicorn 프로덕션 서버를 실행하는 명령
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "bookr:wsgi"]
```

서비스로서의 플랫폼

서비스로서의 플랫폼 (PaaS) 제공업체는 시스템 관리에 대한 배경 지식 없이도 활용할 수 있는 호스팅 환경을 제공합니다. 이들은 Django 프로젝트와 그 보조 서비스(예: 데이터베이스 서버, 백업 및 정적 파일 호스팅)를 실행할 수 있는 Python 환경을 호스팅하는 클라우드 플랫폼입니다.

인기 있는 초급 PaaS 제공업체로는 Render.com, PythonAnywhere.com, Seenode.com 및 Railway.com이 있습니다. 기존 클라우드 제공업체인 Digital Ocean, AWS 및 Google Cloud Platform도 PaaS 구성에 맞춰 서비스를 제공합니다.

Many of these providers have detailed HOWTO documents and videos on deploying Django projects on their platforms. In the following exercise, we will conduct a walkthrough of a Django deployment with the Render.com PaaS.

Exercise 15.02 – Deploying the Bookr project

In this exercise, we will work through the steps involved in the deployment of a Django project to a PaaS environment. We will be creating a repository for the Bookr project using GitHub.com and then using Render.com to host it. This is a simple deployment strategy that involves leveraging web services and doesn't require a DevOps or system admin background.

Note

At the time of writing, Render.com has a free tier in its pricing structure. There are some limitations with this option, as it doesn't give command-line access, file storage is not persistent, and the database is reset after 30 days, with the loss of data.

There is information on the pricing tiers here at <https://render.com/pricing>.

In order to follow these instructions, you will need to create a `github.com` account if you do not have one already. The exercise has been structured so that no knowledge of the command-line Git (or a GUI) is required. However, if you have expertise with Git, you may create the repository in the way that you are most familiar with.

Note

It is assumed that you have followed the code examples in this book and have a working version of the Django Bookr project on your local computer.

If you don't, and you want to follow the deployment, you can download the code from this path in the `Chapter15` folder in this book's GitHub repo. This version was developed with Python 3.12.4.

Log in to `github.com` and create a new repository by selecting **New repository** from the + menu, as in *Figure 15.3*.

이 제공업체들 중 많은 곳이 그들의 플랫폼에서 Djangoprojects를 배포하는 방법에 대한 자세한 HOWTO 문서와 비디오를 가지고 있습니다. 다음 연습에서는 Render.com PaaS를 사용한 Django 배포 과정을 안내할 것입니다.

연습 15.02 – Bookr 프로젝트 배포

이 연습에서는 PaaS idx_55efe475 환경에 Django 프로젝트를 배포하는 데 필요한 단계를 진행할 것입니다. 우리는 GitHub.com을 사용하여 Bookr 프로젝트를 위한 idx_a0bbc7bd 저장소를 만들고, 그 후 Render.com을 사용하여 호스팅할 것입니다. 이는 웹 서비스를 활용하는 간단한 배포 전략으로, DevOps나 시스템 관리자의 배경이 필요하지 않습니다.

참고

작성 시점에서 Render.com은 가격 구조에 무료 계층을 가지고 있습니다. 이 옵션에는 몇 가지 제한이 있으며, 명령줄 접근이 제공되지 않고, 파일 저장소는 지속적이지 않으며, 데이터베이스는 30일 후에 초기화되어 데이터가 손실됩니다.

가격 계층에 대한 정보는 <https://render.com/pricing> 에서 확인할 수 있습니다.

이 지침을 따르기 위해서는 github.com 계정이 필요합니다. 연습은 명령줄 Git(또는 GUI)에 대한 지식이 필요하지 않도록 구성되었습니다. 그러나 Git에 대한 전문 지식이 있는 경우, 가장 익숙한 방법으로 저장소를 만들 수 있습니다.

참고

이 책의 코드 예제를 따라 Django Bookr 프로젝트의 작동 버전을 로컬 컴퓨터에 설치했다고 가정합니다.

그렇지 않다면 배포를 따르려면 이 책의 GitHub 저장소의 Chapter15 폴더에서 이 경로의 코드를 다운로드할 수 있습니다. 이 버전은 Python 3.12.4로 개발되었습니다.

github.com에 로그인하고 + 메뉴에서 새 저장소를 선택하여 새 저장소를 만듭니다. 그림 15.3과 같습니다.

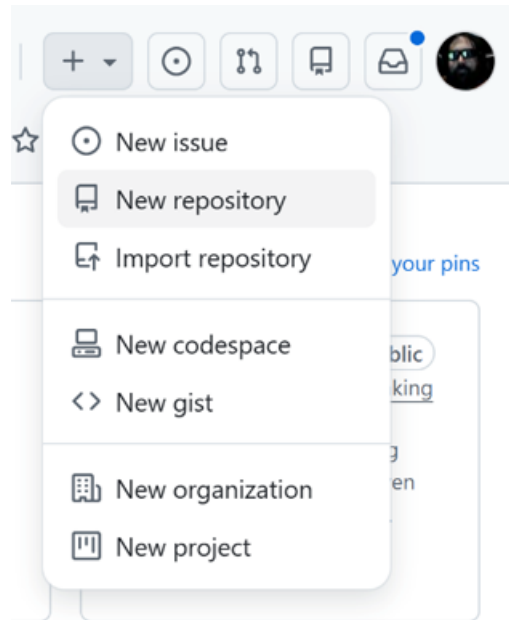


Figure 15.3: Adding a new repository in GitHub

Now, we see a blank form where we enter the details for our new repository, as in *Figure 15.4*.

In the **General** section of the form, we can give the repository a name such as `bookr`. In the **Configuraton** section, do the following:

Set **Choose visibility** to **Public** (this simplifies the `render.com` deployment process)

For **Add .gitignore**, select **Python** from the dropdown.

Now, we can click the **Create repository** button at the bottom of the screen.

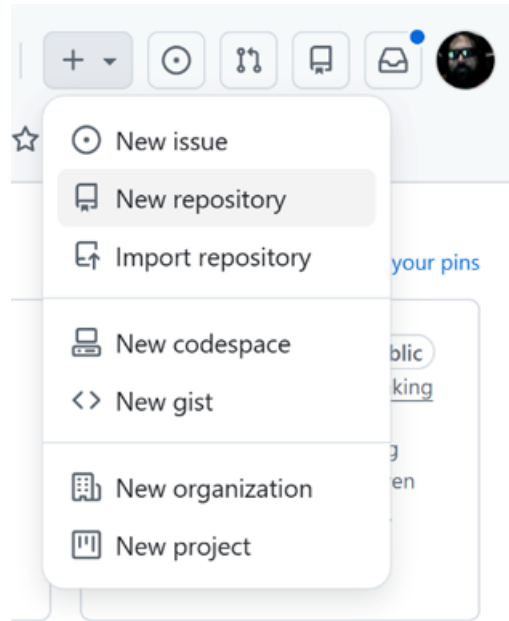


그림 15.3: GitHub에 새 리포지토리 추가하기

이제 우리는 그림 15.4와 같이 새 리포지토리에 대한 세부 정보를 입력하는 `idx_c107ad61a` 빈 양식을 봅니다. 양식의 일반 섹션에서 `bookr`와 같은 이름을 리포지토리에 부여할 수 있습니다. 구성 섹션에서 다음을 수행합니다:

가시성 선택을 공개로 설정합니다(이것은 `render.com` 배포 프로세스를 단순화합니다). `.gitignore` 추가에서 드롭다운에서 Python을 선택합니다. 이제 화면 하단의 리포지토리 생성 버튼을 클릭할 수 있습니다.

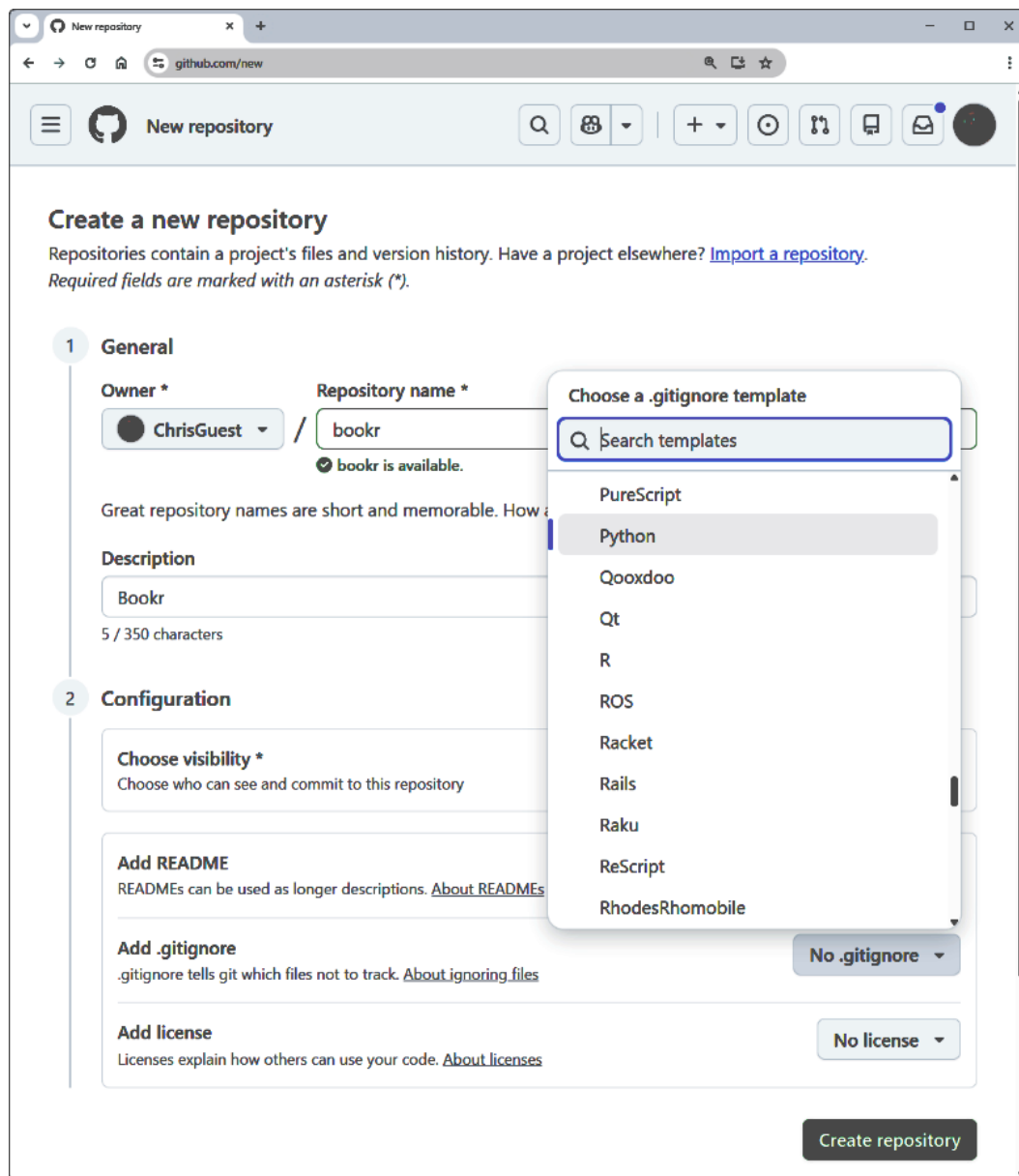


Figure 15.4: Creating a new repository

This will create a repository with a single file called `.gitignore`. This file tells Git which types of files shouldn't be tracked in the repository. The Python `.gitignore` lists SQLite database files, `__pycache__` directories, and a variety of other system files that are generated by the

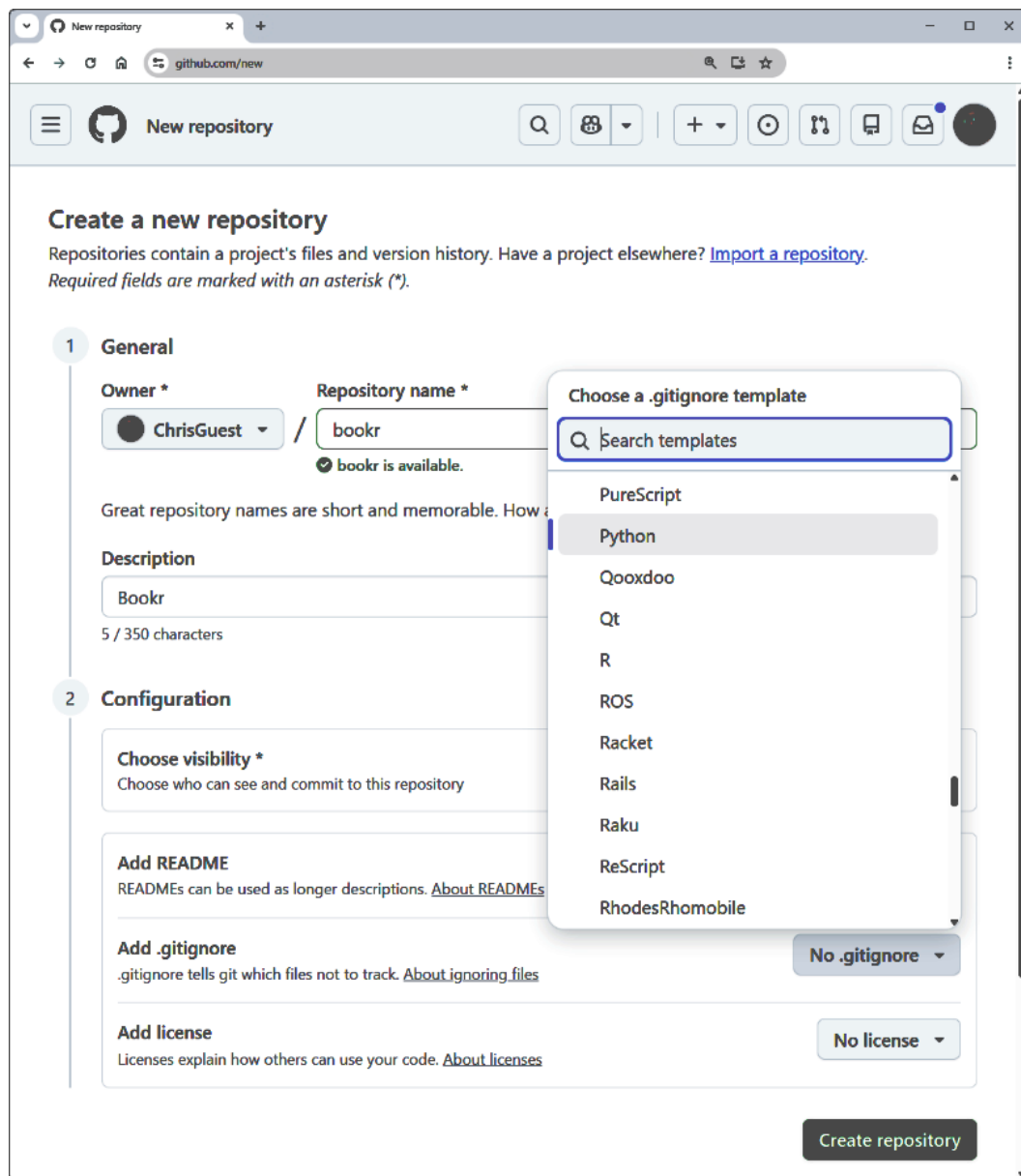


그림 15.4: 새로운 저장소 만들기

이것은 .gitignore라는 단일 파일이 있는 저장소를 생성합니다. 이 파일은 Git에게 저장소에서 추적하지 않아야 할 파일 유형을 알려줍니다. Python .gitignore는 SQLite 데이터베이스 파일, __pycache__ 디렉토리 및 생성되는 다양한 다른 시스템 파일을 나열합니다.

runtime environment or operating system but shouldn't be maintained as part of the code repository.

If we select the **Upload files** option from the + menu at the top of *Figure 15.5*, we get to a page with a drop zone for drag and drop functionality, as in *Figure 15.6*.

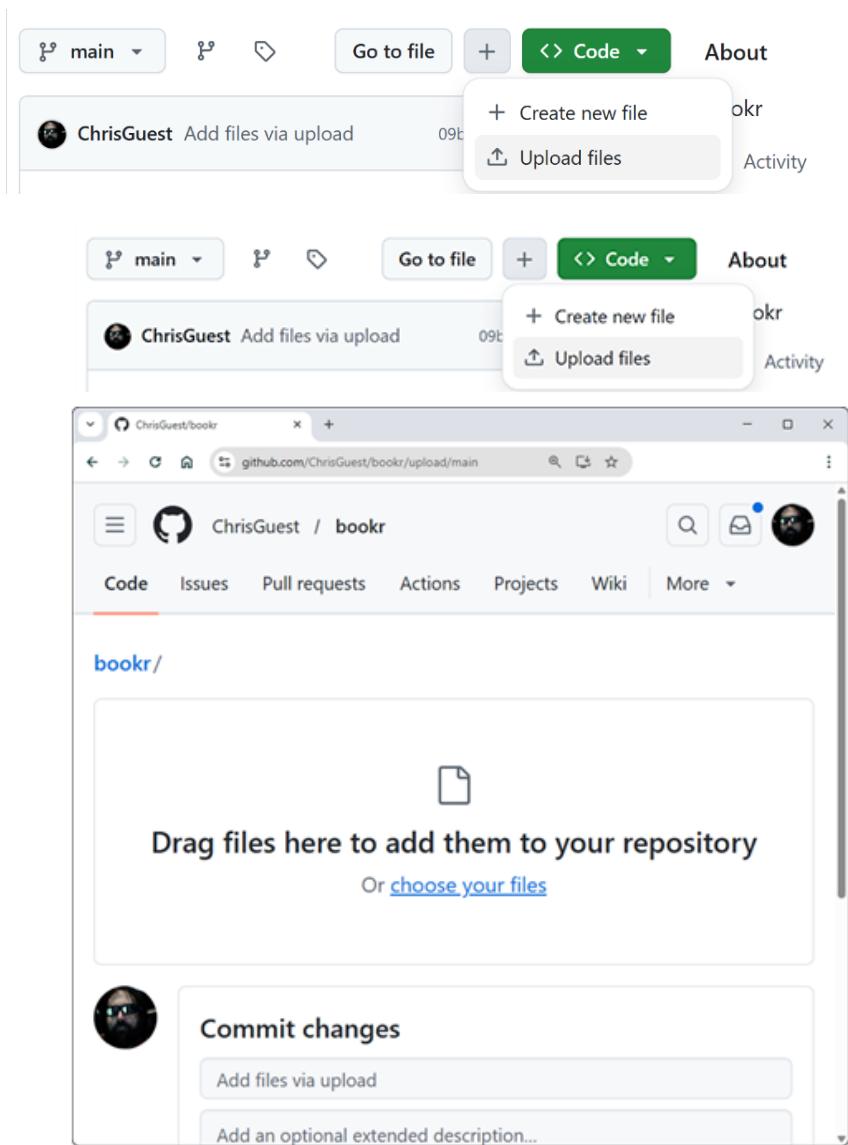


Figure 15.6: The GitHub drop zone

런타임 환경 또는 운영 체제이지만 코드 저장소의 일부로 유지되어서는 안 됩니다.

그림 15.5의 상단에 있는 + 메뉴에서 파일 업로드 옵션을 선택하면 그림 15.6과 같이 드래그 앤 드롭 기능을 위한 드롭 존이 있는 페이지로 이동합니다.

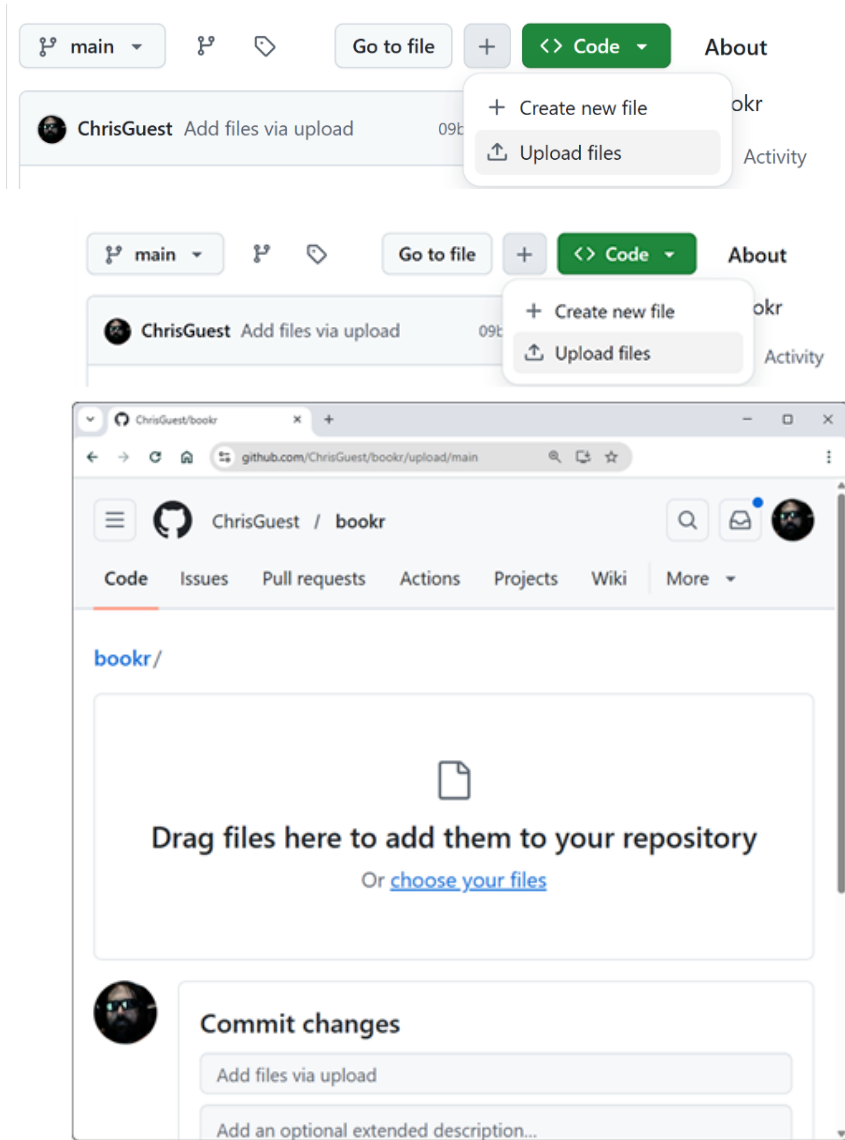


그림 15.6: GitHub 드롭 존

In our Finder or File Explorer, we can browse inside the bookr project folder, selecting all relevant files and folders except the `db.sqlite3` file, `__pycache__`, or the `.env` file, as in *Figure 15.7*. These files can be dragged into the drop zone marked **Drag files here to add them to your repository**.

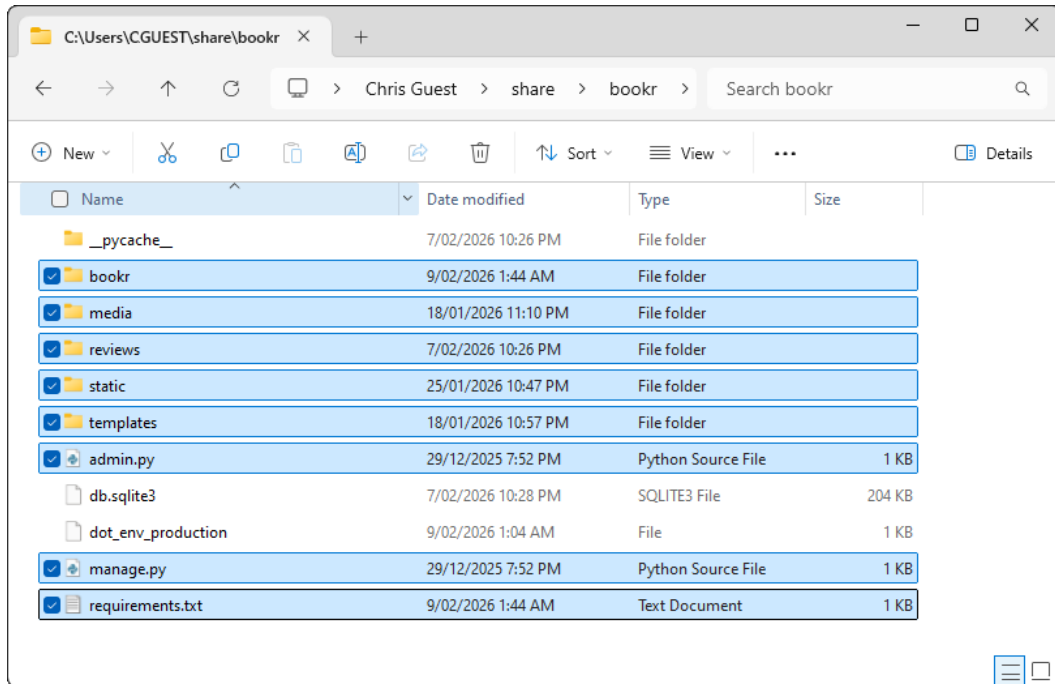


Figure 15.7: The selected files to be dragged and dropped from the filesystem

Now, we can see the files that have been added to this repository in *Figure 15.8*. Although we have a `.gitignore` file, this drag-and-drop step circumvents it, and some unwanted files will have been included, such as the `.pyc` files depicted. These can be manually deleted from the Git commit by clicking the icon link next to each file.

우리의 idx_4a9ca771Finder 또는 파일 탐색기에서, db.sqlite3 파일, __pycache__ 또는 .env 파일을 제외한 모든 관련 파일과 폴더를 선택하여 bookr 프로젝트 폴더 안을 탐색할 수 있습니다. 그림 15.7과 같이 이 파일들은 드래그하여 '여기에 파일을 드래그하여 추가하세요'라는 드롭 존으로 옮길 수 있습니다.

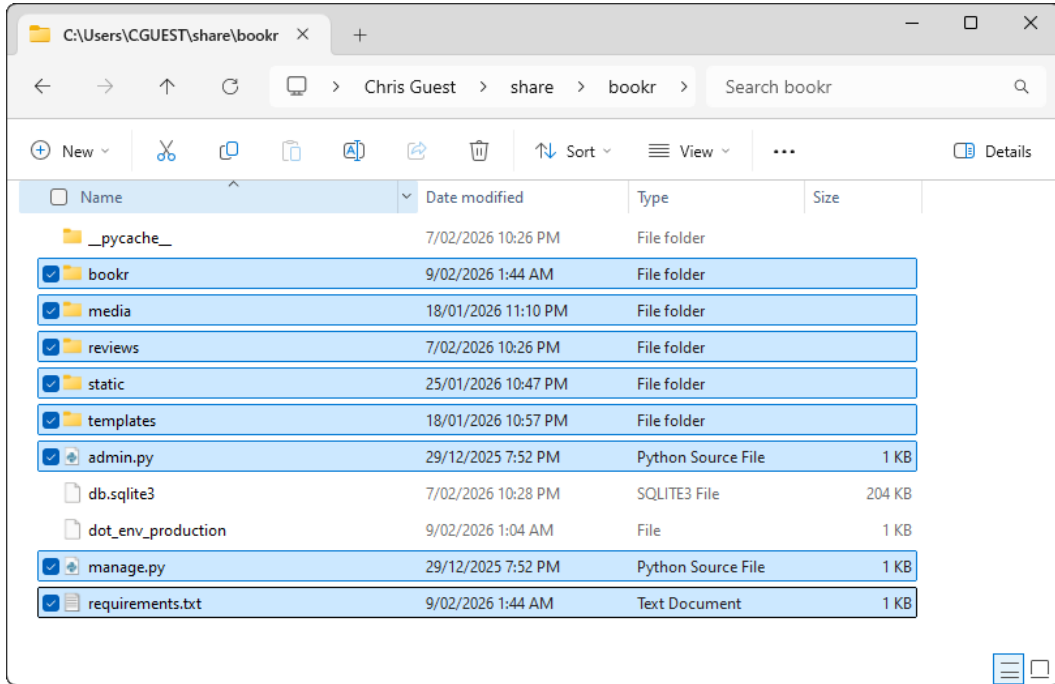


그림 15.7: 파일 시스템에서 드래그 앤 드롭할 선택된 파일들

이제 우리는 그림 15.8에서 이 리포지토리에 추가된 파일들을 볼 수 있습니다. .gitignore 파일이 있지만, 이 드래그 앤 드롭 단계는 이를 우회하며, .pyc 파일과 같이 원하지 않는 파일들이 포함될 수 있습니다. 이러한 파일들은 각 파일 옆의 아이콘 링크를 클릭하여 Git 커밋에서 수동으로 삭제할 수 있습니다.

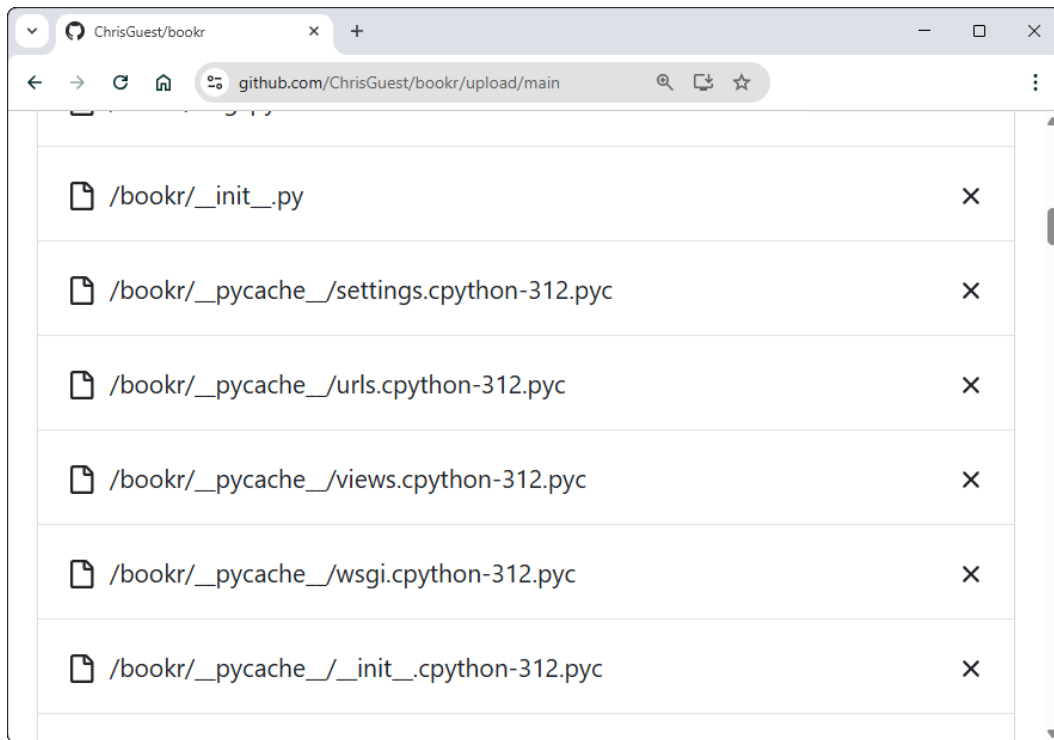


Figure 15.8: The files added to the GitHub repository

At the bottom of this page is the **Commit changes** dialog. We can add a commit message and an optional extended description before clicking the green **Commit changes** button.

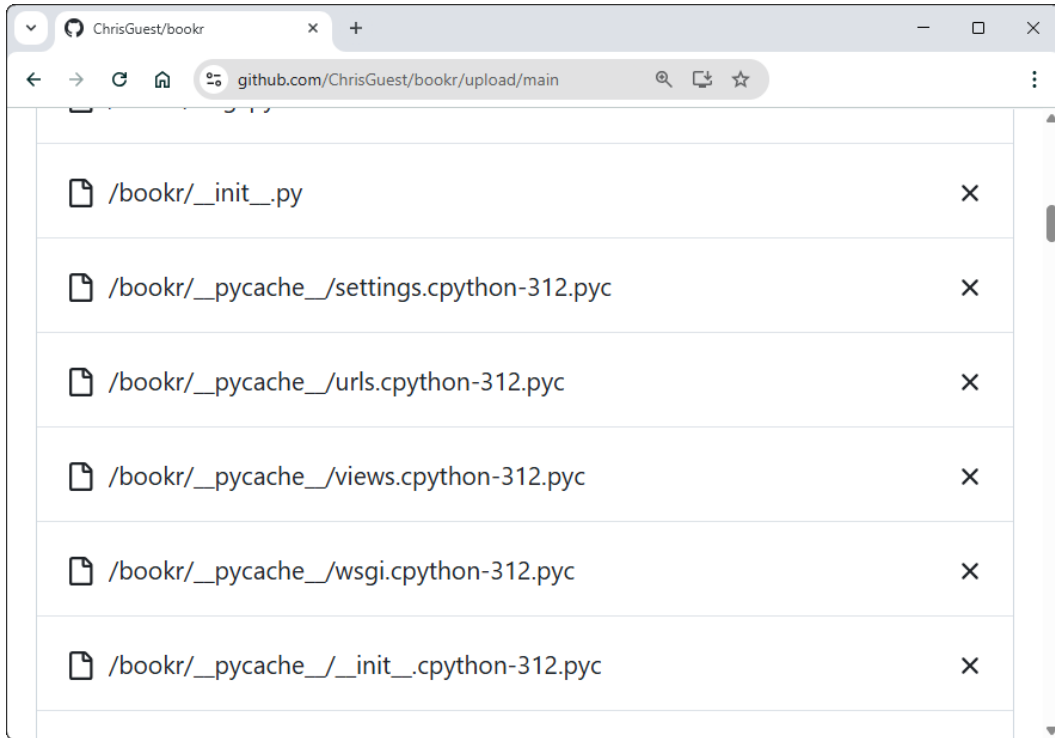


그림 15.8: GitHub 리포지토리에 추가된 파일들

이 페이지 하단에는 커밋 변경 대화 상자가 있습니다. 우리는 커밋 메시지와 선택적인 확장 설명을 추가한 후 녹색 커밋 변경 버튼을 클릭할 수 있습니다.

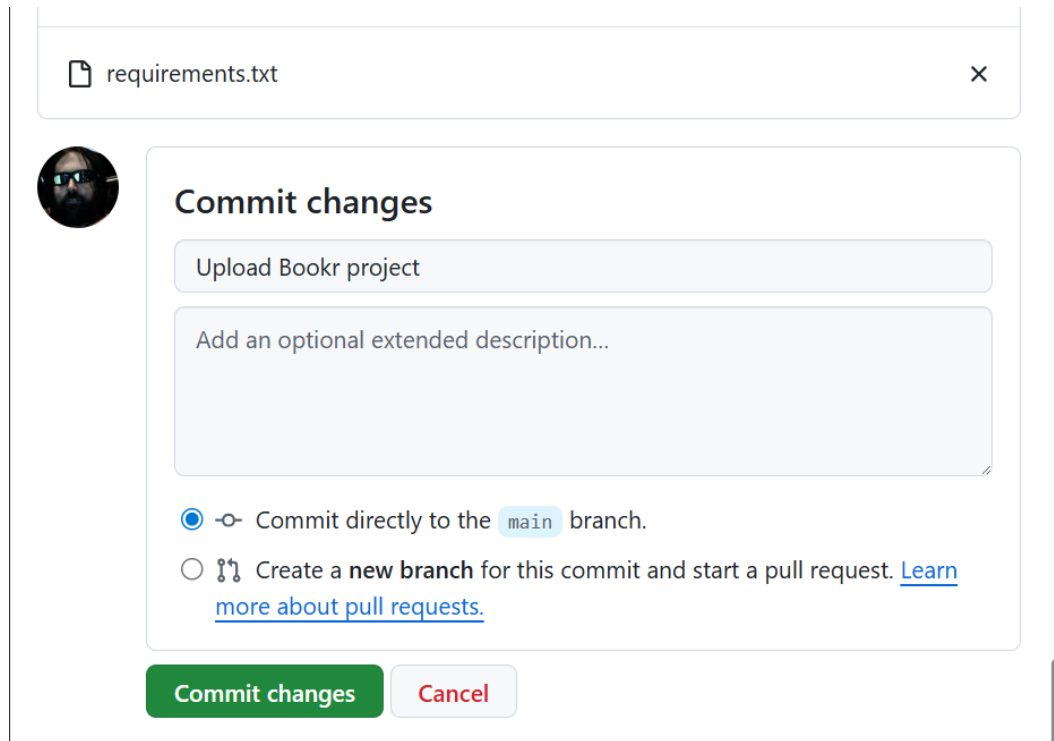


Figure 15.9: The GitHub Commit changes dialog

We have now loaded our Bookr project into a Git repository called bookr that will be publicly visible at <https://github.com/<username>/bookr>, as per *Figure 15.10*.

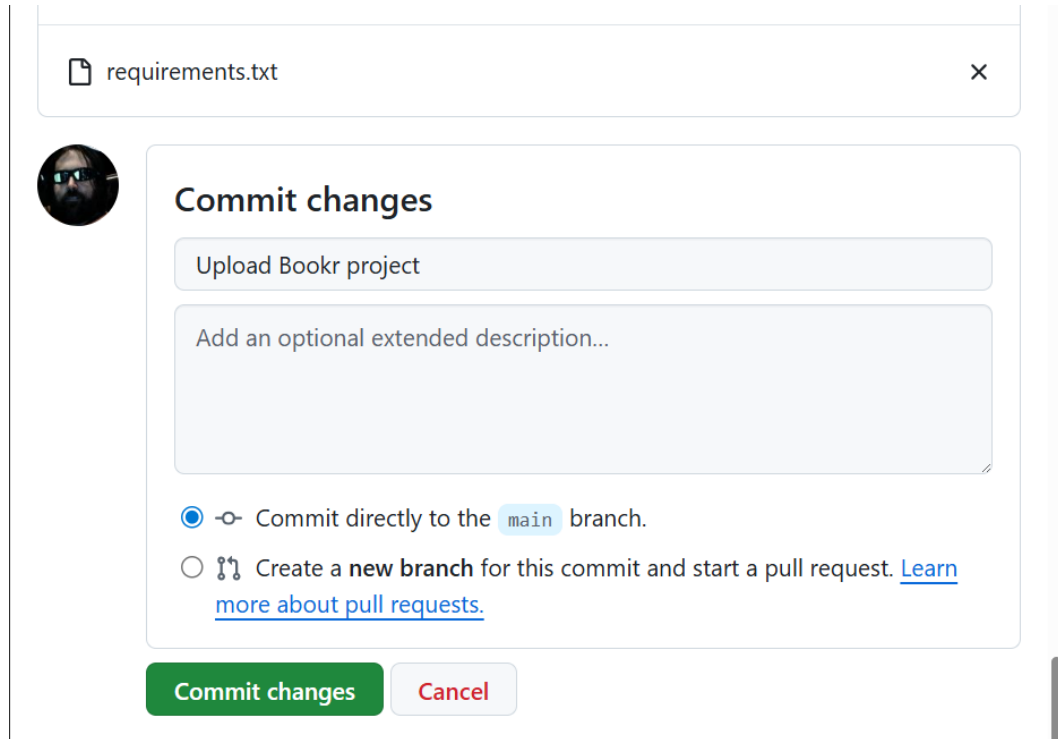


그림 15.9: GitHub 커밋 변경 대화상자

우리는 idx_481de097 이제 <https://github.com/<username>/bookr>에서 공개적으로 볼 수 있는 bookr라는 Git 리포지토리에 우리의 Bookr 프로젝트를 로드했습니다. 그림 15.10에 따라.

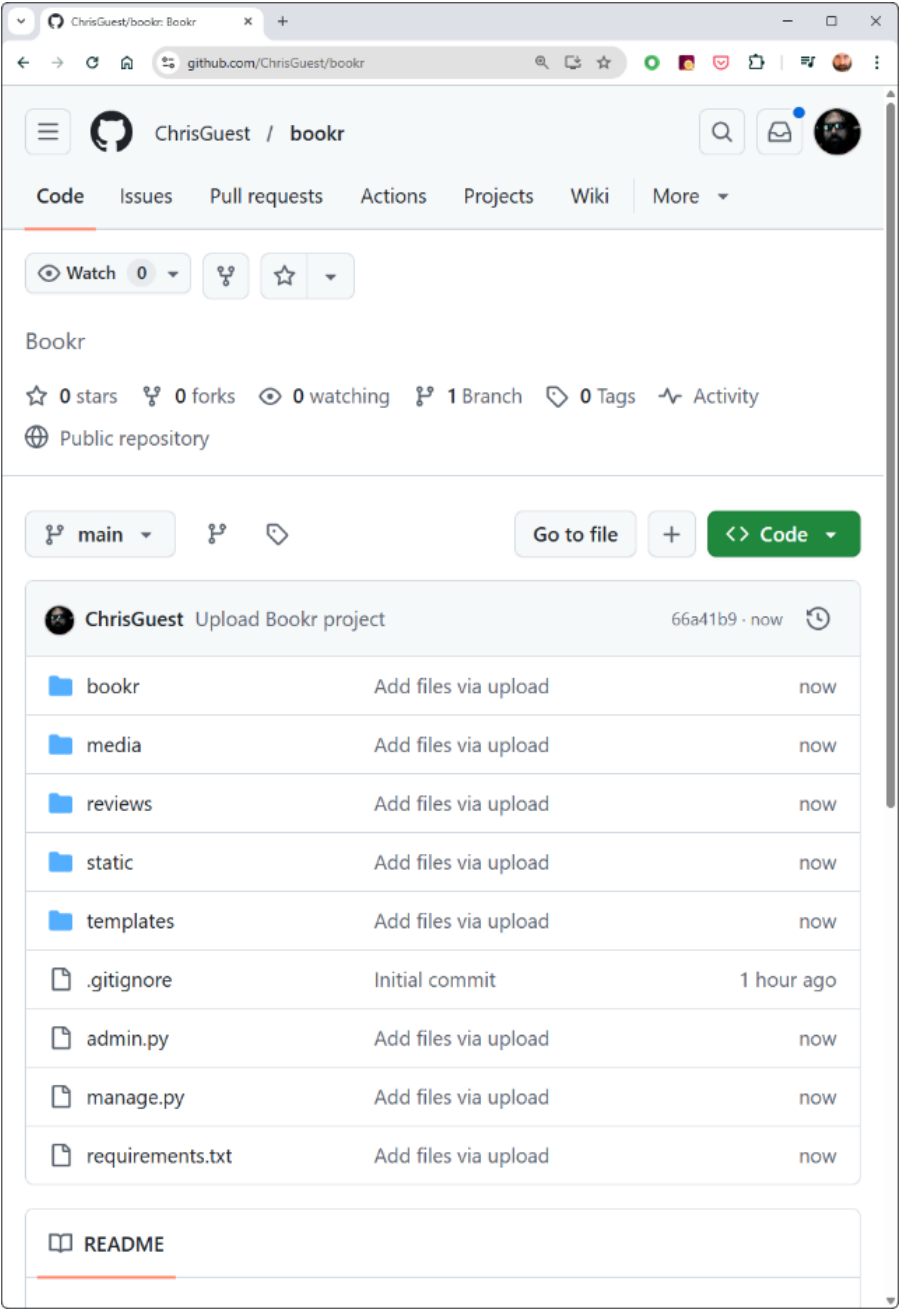


Figure 15.10: The repository containing the Bookr project

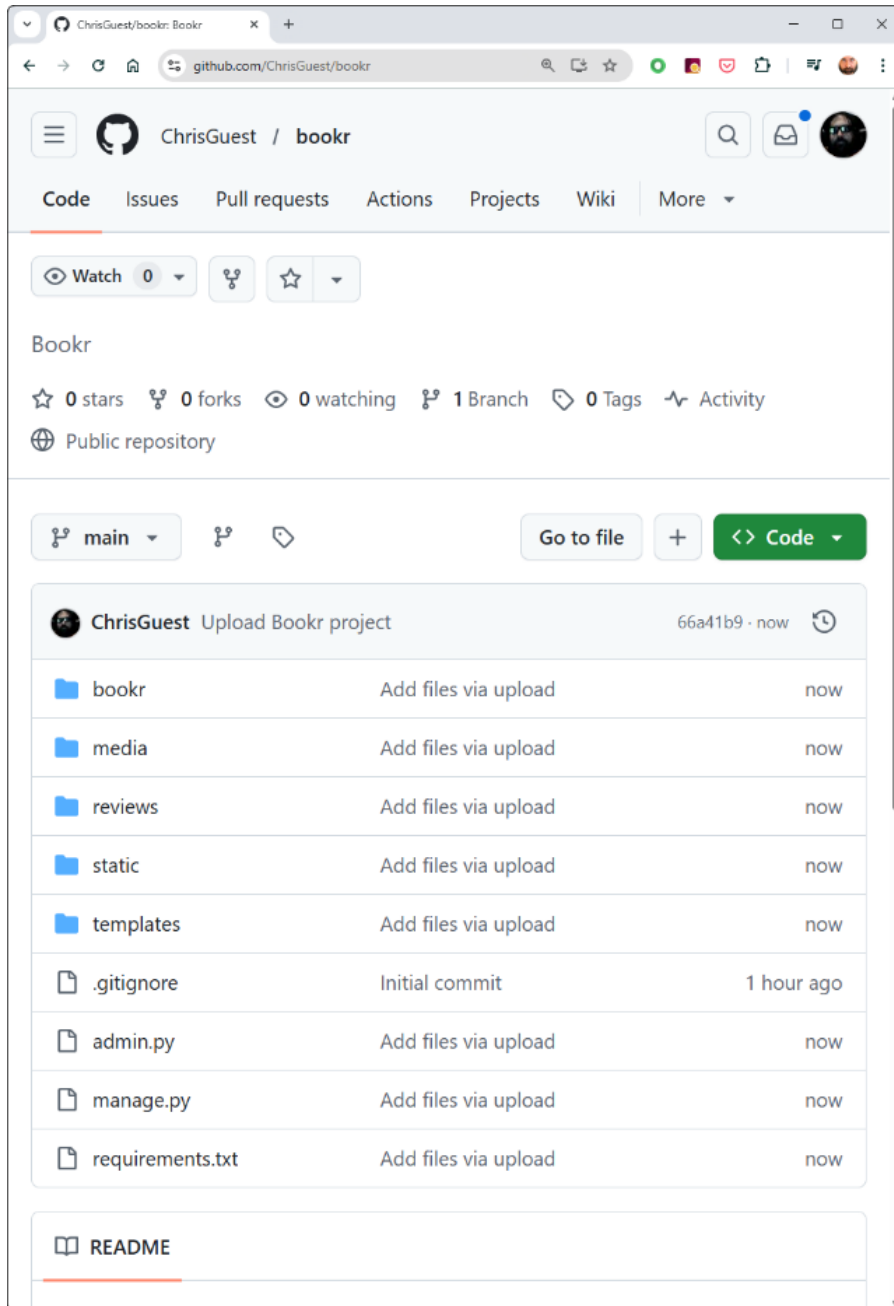


그림 15.10: Bookr 프로젝트를 포함하는 저장소

Now that we have our repository built, we can create an account on `render.com` and build the necessary services for the deployment. We can use the free account tier, **Hobby - For hobbyists and students**. After you have created your account, you will see the dashboard where you can create new services.

Render.com offers **Static Sites**, **Web Services**, **Private Services**, **Background Workers**, **Cron Jobs**, **Postgres**, and **Key Value** as its PaaS components. For this exercise, we will be deploying Postgres and a web service.

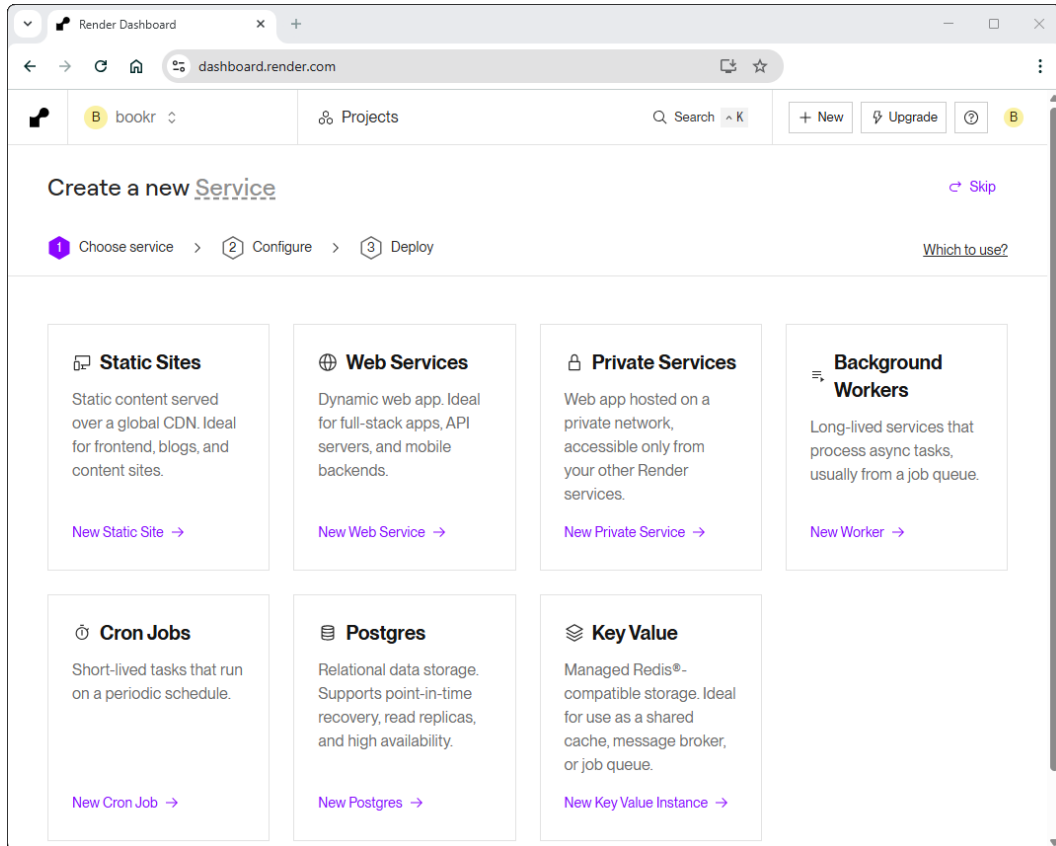


Figure 15.11: The Render.com dashboard

Select **New Postgres** from the Render dashboard. You will see a form to configure and deploy your new database, as in *Figure 15.12*. Give an appropriate instance name and database name, such as `pg-bookr` and `bookr`, respectively. Select a region that you expect will be closest to your user base. At present, Render offers hosting sites in North America, Europe, and Asia.

이제 idx_da0e90e6we 저장소가 구축되었으므로 render.com에서 계정을 만들고 배포를 위한 필요한 서비스를 구축할 수 있습니다. 무료 계정 등급인 Hobby - 취미 및 학생을 사용할 수 있습니다. 계정을 만든 후에는 새로운 서비스를 생성할 수 있는 대시보드가 표시됩니다.

Render.com은 정적 사이트, 웹 서비스, 개인 서비스, 백그라운드 작업자, 크론 작업, Postgres 및 Key Value를 idx_8a024487PaaS 구성 요소로 제공합니다. 이 연습에서는 Postgres와 웹 서비스를 배포할 것입니다.

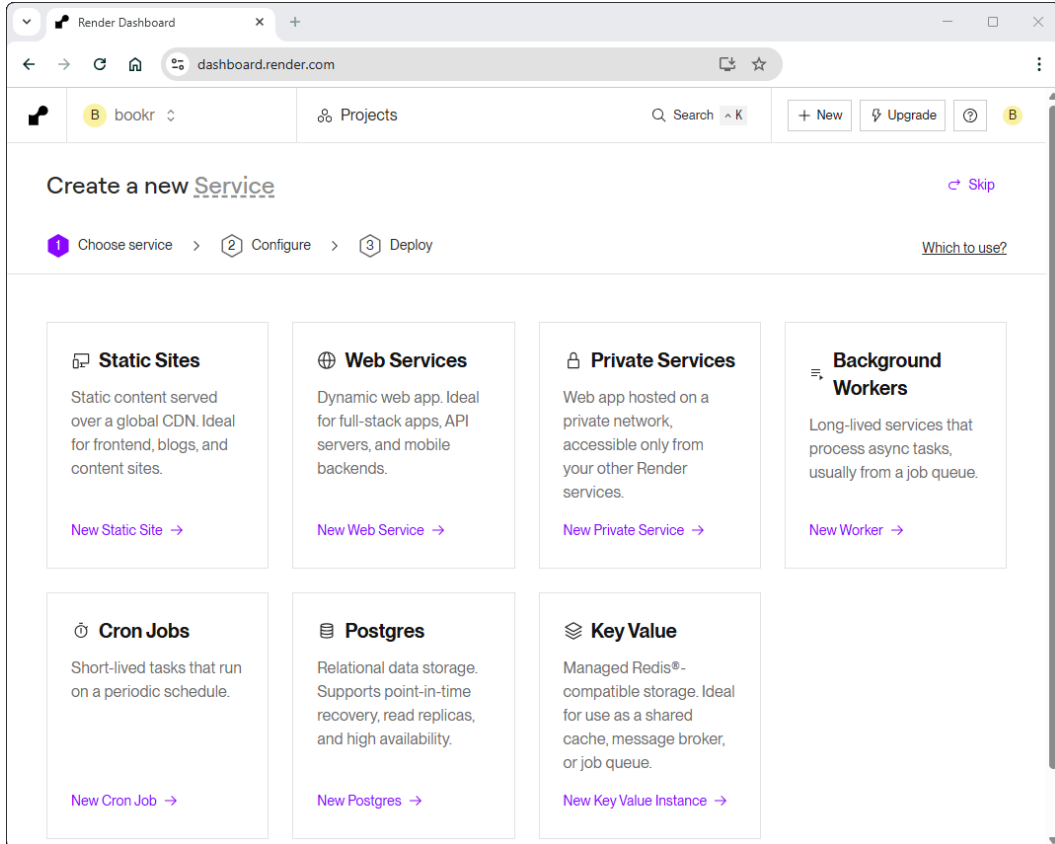
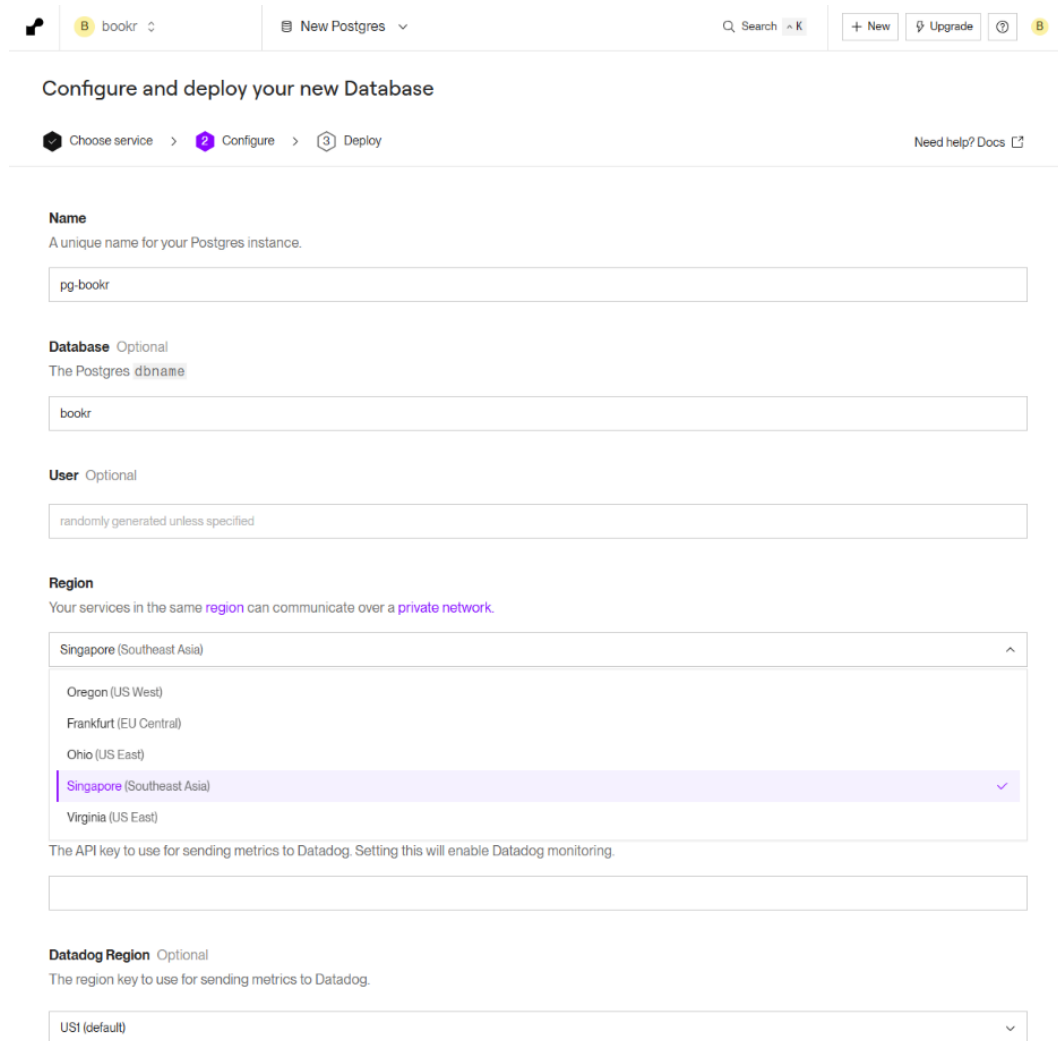


그림 15.11: Render.com 대시보드

Render 대시보드에서 New Postgres를 선택합니다. 그림 15.12와 같이 새 idx_0f632aa2 데이터베이스를 구성하고 배포하기 위한 양식이 표시됩니다. 적절한 인스턴스 이름과 데이터베이스 이름을 지정하십시오. 예를 들어 pg-bookr 및 bookr와 같이 지정합니다. 사용자 기반에 가장 가까울 것으로 예상되는 지역을 선택하십시오. 현재 Render는 북미, 유럽 및 아시아에 호스팅 사이트를 제공합니다.



Configure and deploy your new Database

Choose service > **Configure** > Deploy Need help? Docs

Name
A unique name for your Postgres instance.

pg-bookr

Database Optional
The Postgres `dbname`

bookr

User Optional
randomly generated unless specified

Region
Your services in the same [region](#) can communicate over a [private network](#).

Singapore (Southeast Asia) ^

Oregon (US West)

Frankfurt (EU Central)

Ohio (US East)

Singapore (Southeast Asia) ✓

Virginia (US East)

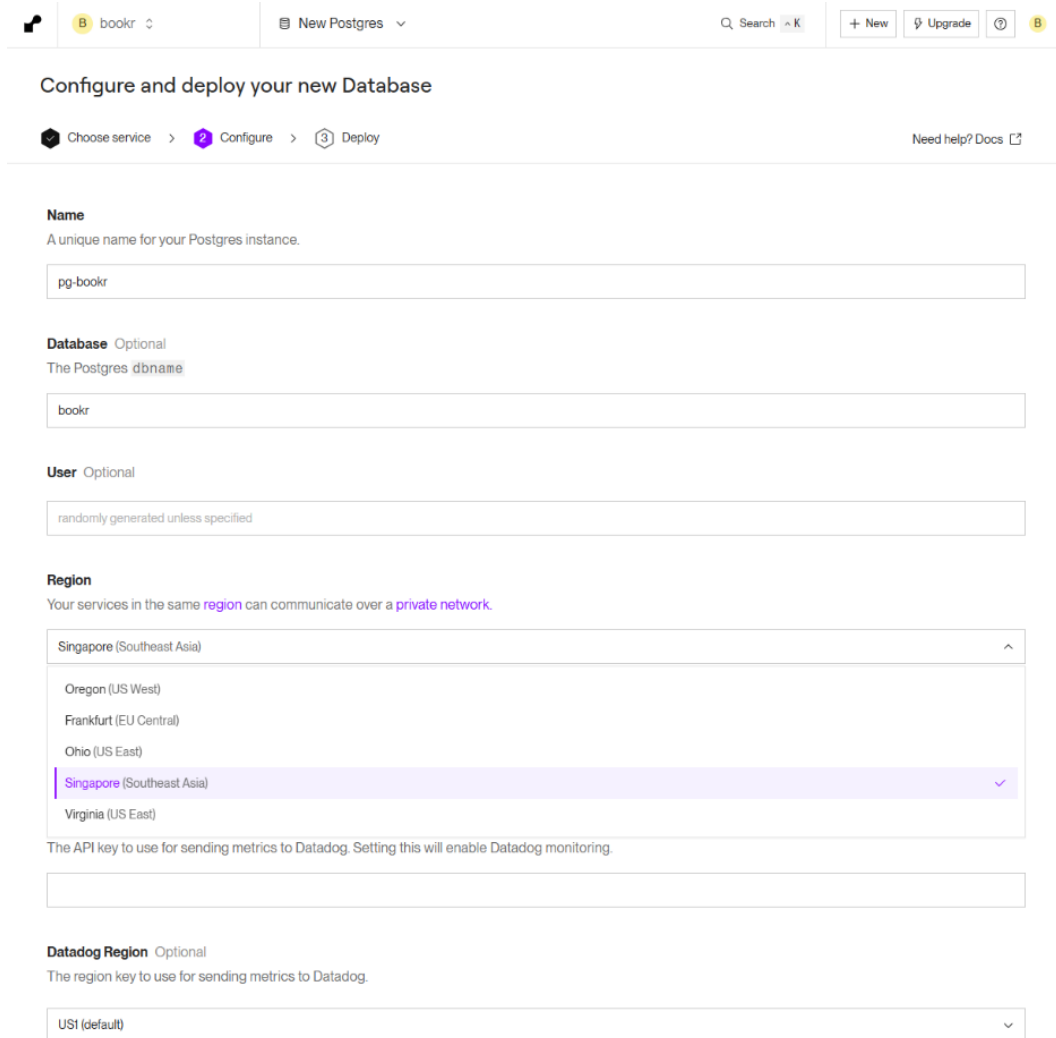
The API key to use for sending metrics to Datadog. Setting this will enable Datadog monitoring.

Datadog Region Optional
The region key to use for sending metrics to Datadog.

US1 (default) v

Figure 15.12: The new database form

For now, we can scroll down and select the **Free** instance type. At the time of writing, Render does not persist Free Postgres instances after the 30-day trial period and does not give access to their hosted backup services. A Postgres instance can be upgraded to a paying tier during this time. The Free tier currently offers 1 GB of storage, which is sufficient for an initial deployment of many applications.



Configure and deploy your new Database

Choose service > **Configure** > Deploy Need help? Docs

Name
A unique name for your Postgres instance.

pg-bookr

Database Optional
The Postgres dbname

bookr

User Optional
randomly generated unless specified

Region
Your services in the same region can communicate over a private network.

Singapore (Southeast Asia) ^

Oregon (US West)

Frankfurt (EU Central)

Ohio (US East)

Singapore (Southeast Asia) v

Virginia (US East)

The API key to use for sending metrics to Datadog. Setting this will enable Datadog monitoring.

Datadog Region Optional
The region key to use for sending metrics to Datadog.

US1 (default) v

그림 15.12: 새로운 데이터베이스 양식

현재로서는 idx_7b052017 아래로 스크롤하여 무료 인스턴스 유형을 선택할 수 있습니다. 작성 시점에서 Render는 30일 시험 기간 후 무료 Postgres 인스턴스를 유지하지 않으며 호스팅된 백업 서비스에 대한 액세스를 제공하지 않습니다. 이 기간 동안 Postgres 인스턴스를 유료 계층으로 업그레이드할 수 있습니다. 무료 계층은 현재 1GB의 저장소를 제공하며, 이는 많은 애플리케이션의 초기 배포에 충분합니다.

Plan Options

Instance Type

Set your database's RAM and CPU. You can change your instance type later.

Storage

Your database's capacity, in GB. You can increase storage later, but you can't decrease it.
Specify 1 GB or any multiple of 5 GB.

Storage Autoscaling

When your database is 90% full, automatically increase its storage by 50% (rounded up to the nearest 5 GB).
Max one increase per 12 hours. [Learn more](#)

High Availability

Run a standby instance of your database and automatically fail over to it if the primary encounters an issue. [Learn more](#)

☒ Free

For testing out Render Postgres

Free

\$0 / month

256 MB (RAM)

0.1 CPU

1 GB (Storage)

☐ Basic

Reliability and performance for hobby projects. Starting at \$6 / month plus storage.

☐ Pro

Perfect for production use cases at any scale. Starting at \$55 / month plus storage.

☐ Accelerated

Memory-optimized plans offer significant performance improvements. Starting at \$160 / month plus storage.

Need a [custom instance type](#)? We support up to 1024 GB RAM, 128 CPUs, and 16 TB storage.

1

GB

\$0 / month

☐ Disabled

☐ Disabled

Only available for **Pro** instances and higher.

Figure 15.13: Render.com plan options

Once you have confirmed that the **Monthly Total** cost is expected, click the **Create Database** button.

Monthly Total

Database instances are billed and prorated by the second.

Instance: Free

\$0 / month

Total

\$0 / month

Create Database

Figure 15.14: Render.com monthly total

You will see a summary of the created database instance. Take note of the database expiry date, as you will need to upgrade to a paid tier by this time or risk losing your data.

Plan Options

Instance Type

Set your database's RAM and CPU. You can change your instance type later.

Free

For testing out Render Postgres

Free

\$0 / month

256 MB (RAM)

0.1 CPU

1 GB (Storage)

Basic

Reliability and performance for hobby projects. Starting at \$6 / month plus storage.

Pro

Perfect for production use cases at any scale. Starting at \$55 / month plus storage.

Accelerated

Memory-optimized plans offer significant performance improvements. Starting at \$160 / month plus storage.

Need a [custom instance type](#)? We support up to 1024 GB RAM, 128 CPUs, and 16 TB storage.

Storage

Your database's capacity, in GB. You can increase storage later, but you can't decrease it.

Specify 1 GB or any multiple of 5 GB.

1

GB

\$0 / month

Storage Autoscaling

When your database is 90% full, automatically increase its storage by 50% (rounded up to the nearest 5 GB).

Max one increase per 12 hours. [Learn more](#)

Disabled

High Availability

Run a standby instance of your database and automatically fail over to it if the primary encounters an issue. [Learn more](#)

Disabled

Only available for **Pro** instances and higher.

그림 15.13: Render.com 요금제 옵션

월 총 비용이 예상된다는 것을 확인한 후, 데이터베이스 생성 버튼을 클릭하세요.

Monthly Total

Database instances are billed and prorated by the second.

Instance: Free	\$0 / month
Total	\$0 / month

Create Database

그림 15.14: Render.com 월 총액

생성된 데이터베이스 인스턴스의 요약을 볼 수 있습니다. 데이터베이스 만료 날짜를 기록해 두세요. 이 시점까지 유료 요금제로 업그레이드하지 않으면 데이터 손실 위험이 있습니다.

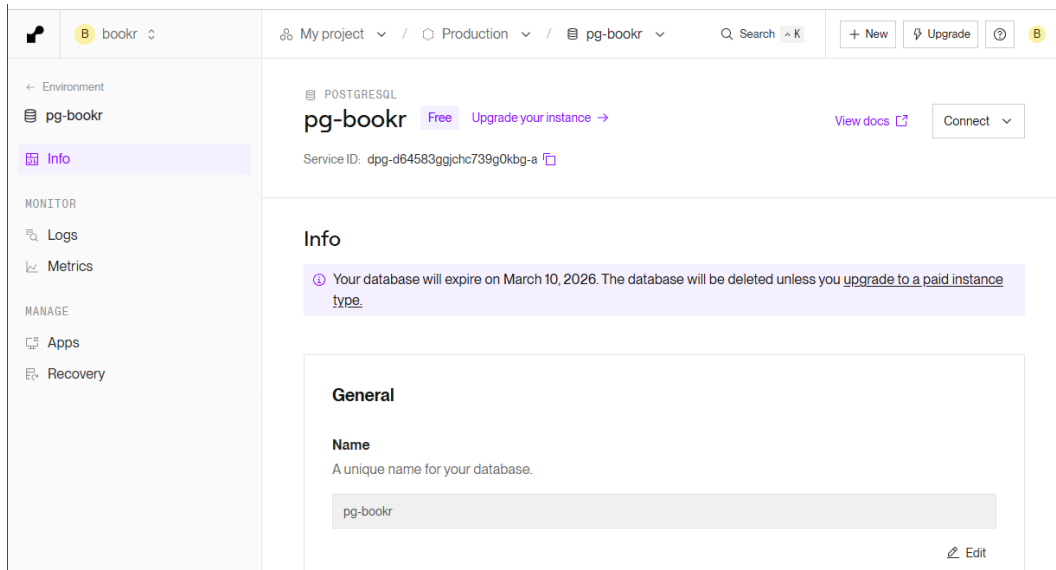


Figure 15.15: PostgreSQL database summary

Hit the *view* and *copy* icons of **Internal Database URL**.



Figure 15.16: Internal database URL

Add this string into the production `.env` file:

```
DATABASE_URL=postgresql://bookr_<...>_user:<long password>/bookr_<...>
```

Our Postgres service is now configured. We can start provisioning the web service.

We need to add a `PYTHON_VERSION` setting to our `.env` file. This reflects the version of Python that our Django project is running with. Render will use this information to provision an appropriate version of Python on the web service:

```
PYTHON_VERSION=3.12.4
```

From the Render dashboard, click **Add Web Service**. You will be taken to a form with web service details. Under **Source Code**, include your Git repository URL. Fill the **Name**, **Language**,

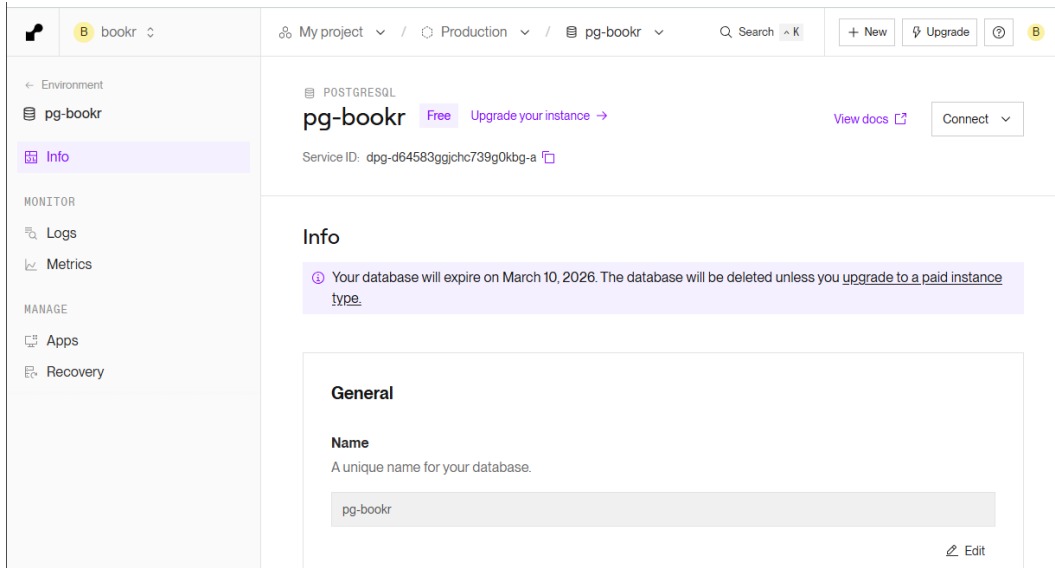


그림 15.15: PostgreSQL 데이터베이스 요약

내부 데이터베이스 URL의 보기 및 복사 아이콘을 클릭하세요.



그림 15.16: 내부 데이터베이스 URL

이 idx_0f9f6c41string을 프로덕션 .env 파일에 추가하세요:

```
DATABASE_URL=postgresql://bookr_<...>_user:<long password>/bookr_<...>
```

우리의 Postgres 서비스가 이제 구성되었습니다. 웹 서비스를 프로비저닝할 수 있습니다.

우리의 .env 파일에 PYTHON_VERSION 설정을 추가해야 합니다. 이는 우리의 장고 프로젝트가 실행되고 있는 Python 버전을 반영합니다. Render는 이 정보를 사용하여 웹 서비스에서 적절한 버전의 Python을 프로비저닝합니다:

```
PYTHON_VERSION=3.12.4
```

Render 대시보드에서 웹 서비스 추가를 클릭하세요. 웹 서비스 세부 정보가 있는 양식으로 이동합니다. 소스 코드 아래에 Git 저장소 URL을 포함하세요. 이름, 언어를 입력하세요,

and **Branch** fields with bookr, Python 3, and main, respectively. Select the same region that you did for the Postgres service.



B

bookr

My project

Production

New Web Service

Search

K

+ New

Upgrade



B

New Web Service

Source Code

Git Provider

Public Git Repository

Existing Image

PR Previews and Auto-Deploy are available only for repositories configured with Blueprints

https://github.com/ChrisGuest/bookr/

Connect

Name

A unique name for your web service.

Bookr

Project

Optional

Add this web service to a project once it's created.

My project

Production

Language

Choose the runtime environment for this service.

Python 3

Branch

The Git branch to build and deploy.

main

Region

Your services in the same region can communicate over a private network. You currently have services running in Singapore.

Singapore (Southeast Asia)

1 existing service

Deploy in a new region

Figure 15.17: The new web service form

그리고 각각 bookr, Python 3 및 main으로 필드를 분기합니다. Postgres 서비스에 대해 선택한 동일한 영역을 선택하십시오.

New Web Service

Source Code

Git Provider **Public Git Repository** Existing Image

PR Previews and Auto-Deploy are available only for repositories configured with Blueprints

<https://github.com/ChrisGuest/bookr/>

Connect →

Name
A unique name for your web service.

Bookr

Project Optional
Add this web service to a **project** once it's created.

My project / Production

Language
Choose the **runtime environment** for this service.

Python 3

Branch
The Git branch to build and deploy.

main

Region
Your services in the same **region** can communicate over a **private network**. You currently have services running in **Singapore**.

Singapore (Southeast Asia) 1 existing service

Deploy in a new region +

그림 15.17: 새로운 웹 서비스 양식

Under **Environment Variables**, there is an option to add from `.env`. You will see a dialog box where you can paste your `.env` file. The environment variables will be extracted from the file and displayed in the form, as per *Figure 15.18*.

Variable Name	Value	Action
SECRET_KEY		🗑️
ENVIRONMENT		🗑️
ALLOWED_HOSTS		🗑️
EMAIL_HOST		🗑️
EMAIL_HOST_USER		🗑️
EMAIL_HOST_PASSWORD		🗑️
EMAIL_PORT		🗑️
EMAIL_USE_TLS		🗑️
EMAIL_USE_SSL		🗑️
DATABASE_URL		🗑️
SECURE_SSL_REDIRECT		🗑️
SECURE_HSTS_SECONDS		🗑️
PYTHON_VERSION	3.12.4	🗑️

Figure 15.18: Environment variables

Below **Environment Variables** is a section of the form containing **Build Command** and **Start Command**. Our build command will initially involve statements to install the requirements, create the database structure with the migrate subcommand, collectstatic, and create a superuser. These statements can be concatenated into a single build statement by using the `&&` operator. You can set your password in the statement by using a Linux environment variable, `DJANGO_SUPERUSER_PASSWORD`. Be sure to include your own username and password:

```
pipinstall-
rrequirements.txt&&pythonmanage.pymigrate&&pythonmanage.pycollectstatic--
```

환경 변수 아래에 .env에서 추가할 수 있는 idx_854a48f0an 옵션이 있습니다. .env 파일을 붙여넣을 수 있는 대화 상자가 표시됩니다. 환경 변수는 파일에서 추출되어 도표 15.18에 따라 양식에 표시됩니다.

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

SECRET_KEY		
ENVIRONMENT		
ALLOWED_HOSTS		
EMAIL_HOST		
EMAIL_HOST_USER		
EMAIL_HOST_PASSWORD		
EMAIL_PORT		
EMAIL_USE_TLS		
EMAIL_USE_SSL		
DATABASE_URL		
SECURE_SSL_REDIRECT		
SECURE_HSTS_SECONDS		
PYTHON_VERSION	3.12.4	

+ Add Environment Variable
Add from .env

도표 15.18: 환경 변수

환경 변수 아래에는 빌드 명령 및 시작 명령을 포함하는 양식의 섹션이 있습니다. 우리의 빌드 명령은 처음에 요구 사항을 설치하고, migrate 하위 명령으로 idx_35bcef03 데이터베이스 구조를 생성하고, collectstatic을 수행하고, 슈퍼유저를 생성하는 명령을 포함할 것입니다. 이러한 명령은 && 연산자를 사용하여 단일 빌드 명령으로 연결할 수 있습니다. Linux 환경 변수인 DJANGO_SUPERUSER_PASSWORD를 사용하여 명령문에 비밀번호를 설정할 수 있습니다. 자신의 사용자 이름과 비밀번호를 포함하는 것을 잊지 마세요:

```
pipinstall
rrequirements.txt&&pythonmanage.pymigrate&&pythonmanage.pycollectstatic--
```

```
noinput&&DJANGO_SUPERUSER_PASSWORD='<HardP@ssw0rd!>' python manage.py  
createsuperuser --noinput --username <yourname> --email <yourname@example.com>
```

On subsequent builds after the initial deployment, you can omit the last clause in this build command as the superuser has already been created.

For our environment, the start command will be `gunicorn bookr.wsgi`. Be sure that you included `gunicorn` in the `requirements.txt` file.

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

e.g. `src`

Build Command

Render runs this command to build your app before each deploy.

```
$ pip install -r requirements.txt && python manage.py migrate && python manage.py collectstatic --noinput && DJANGO_SUPERUSER_PASSWORD='Ha...
```

Start Command

Render runs this command to start your app with each deploy.

```
$ gunicorn bookr.wsgi
```

Figure 15.19: Root directory, build, and start commands

Now you can click the **Deploy Web Service** button at the bottom of the form, and the deployment will begin, as in *Figure 15.20*.

```
noinput&&DJANGO_SUPERUSER_PASSWORD='<HardP@ssw0rd!>' python manage.pycreatesuperuser
--noinput --username <yourname> --email <yourname@example.com>
```

초기 배포 이후의 후속 빌드에서는 슈퍼유저가 이미 생성되었으므로 이 빌드 명령의 마지막 절을 생략할 수 있습니다.

우리의 idx_9dc5aea7 환경에서는 시작 명령이 gunicorn bookr.wsgi가 될 것입니다. requirements.txt 파일에 gunicorn이 포함되어 있는지 확인하세요.

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

e.g. src

Build Command

Render runs this command to build your app before each deploy.

```
$ pip install -r requirements.txt && python manage.py migrate && python manage.py collectstatic --noinput && DJANGO_SUPERUSER_PASSWORD='Ha...
```

Start Command

Render runs this command to start your app with each deploy.

```
$ gunicorn bookr.wsgi
```

그림 15.19: 루트 디렉토리, 빌드 및 시작 명령

이제 양식 하단의 웹 서비스 배포 버튼을 클릭할 수 있으며, 그림 15.20과 같이 배포가 시작됩니다.

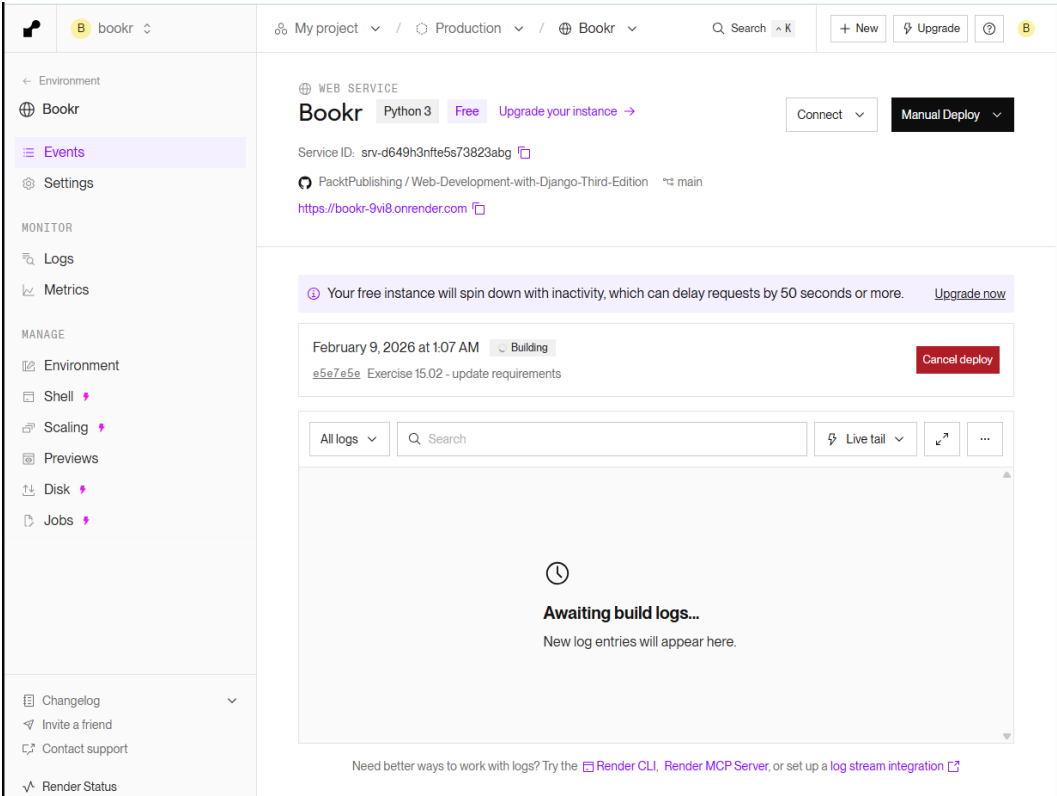


Figure 15.20: Web service build

Note that on this screen, there is a URL given for the target deployment. In the figure given, it is <https://bookr-9vi8.onrender.com>, but it will obviously differ across websites. Visiting this link during the deployment, you will see the service deployment status, as in Figure 15.21.

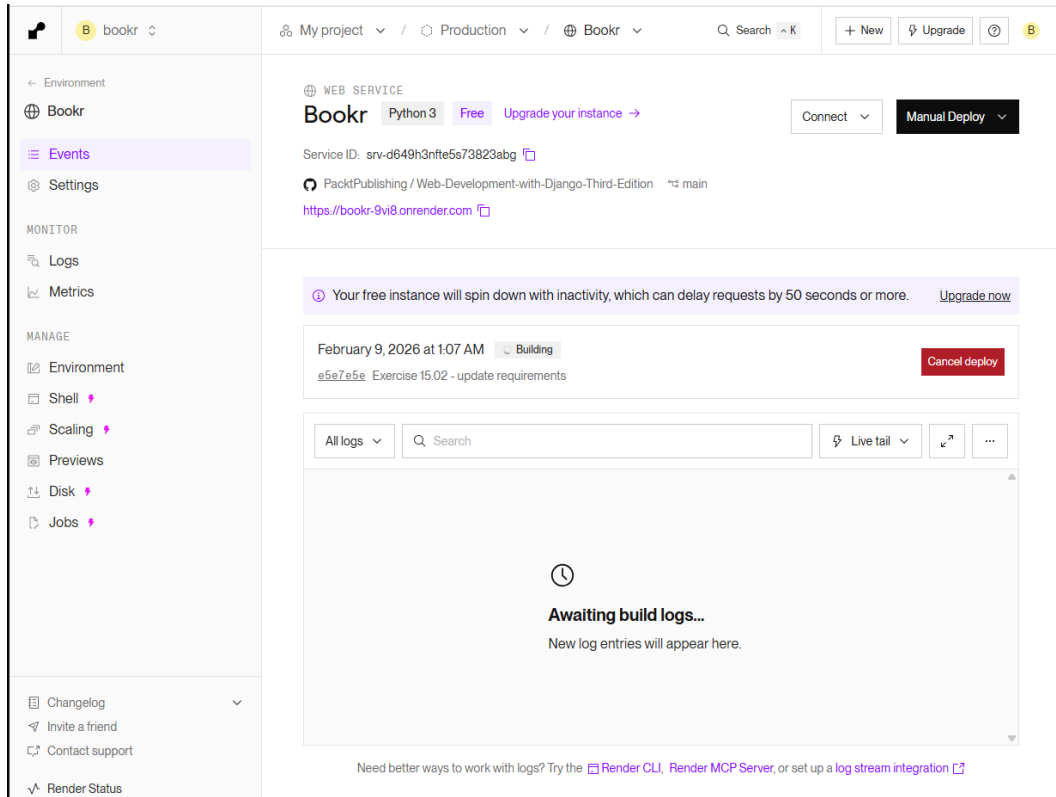


그림 15.20: 웹 서비스 빌드

이 화면에서 대상 배포를 위한 URL이 제공된다는 점에 유의하십시오. 주어진 그림에서는 <https://bookr-9vi8.onrender.com> 이지만, 웹사이트마다 다를 것입니다. 배포 중 이 링크를 방문하면 그림 15.21과 같이 서비스 배포 상태를 볼 수 있습니다.

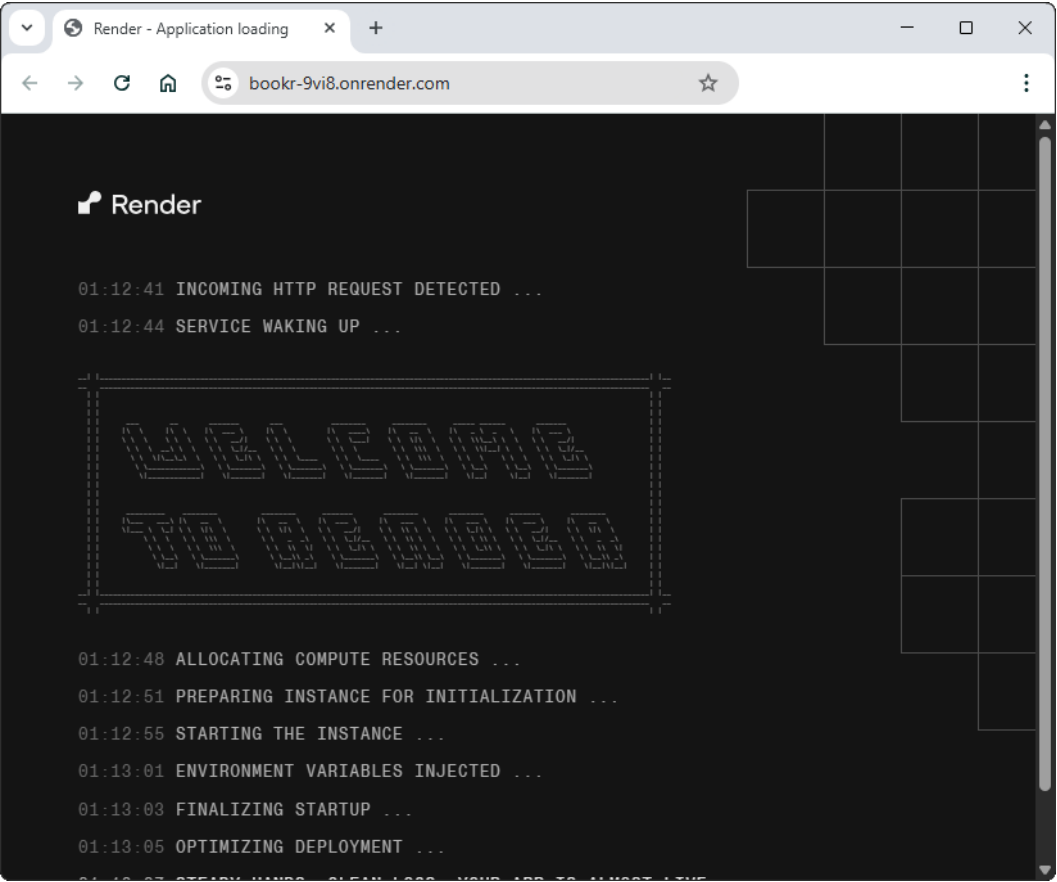


Figure 15.21: Render service deployment status

Eventually, if the deployment is successful, you will see the familiar web page that we have been developing.

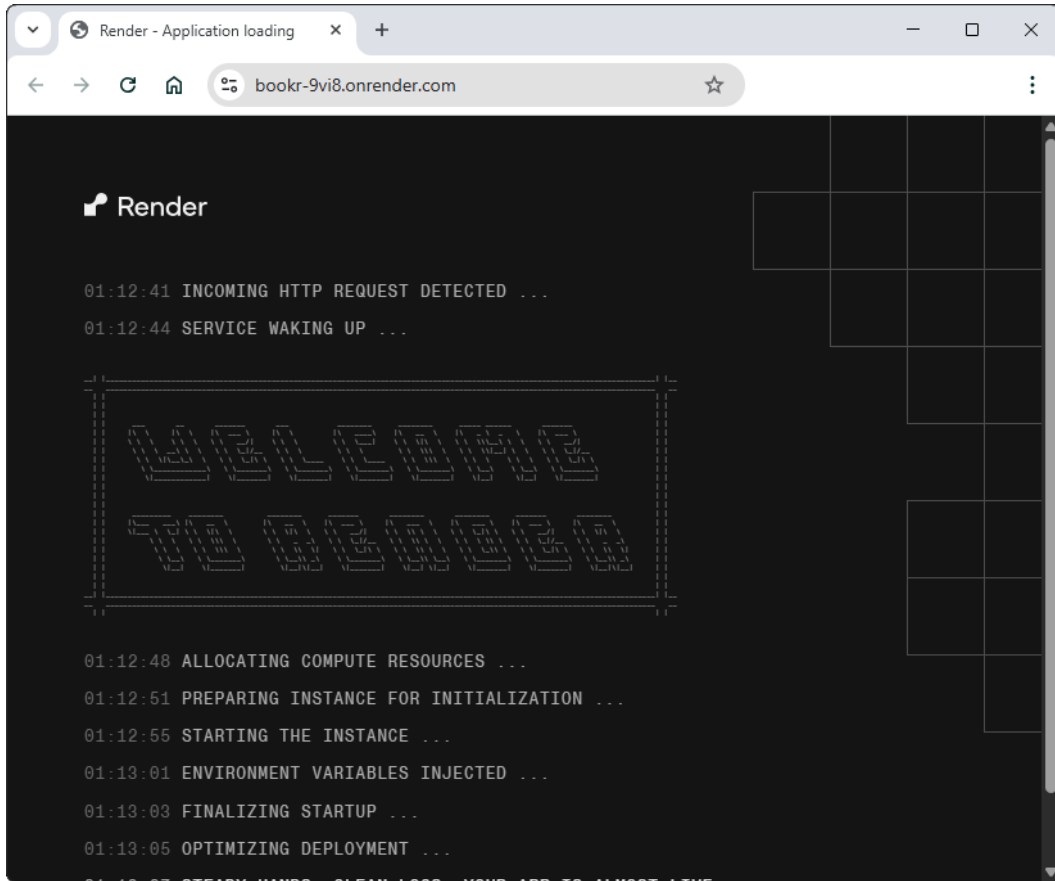


그림 15.21: 렌더 서비스 배포 상태

결국, idx_aca774e8 배포가 성공하면, 우리가 개발해온 익숙한 웹 페이지를 보게 될 것입니다.

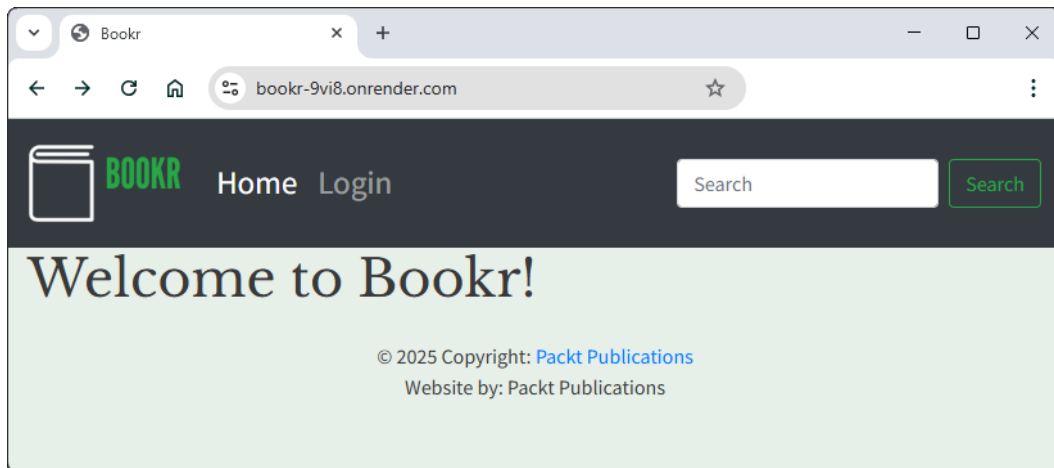


Figure 15.22: The deployed project

Keep an eye on the build screen for any errors. You may have entered a configuration parameter or environment variable incorrectly. There may even be errors in your source code that need to be fixed in the repository before redeploying. This is also the point where we want to add the allocated domain to the `ALLOWED_HOSTS` environment variable rather than use the wildcard:

```
ALLOWED_HOSTS=<yourdomain>.onrender.com
```

When rebuilding after an error or reconfiguration, the safest way to do it is by clicking on **Clear build cache & deploy** in the **Manual Deploy** menu.

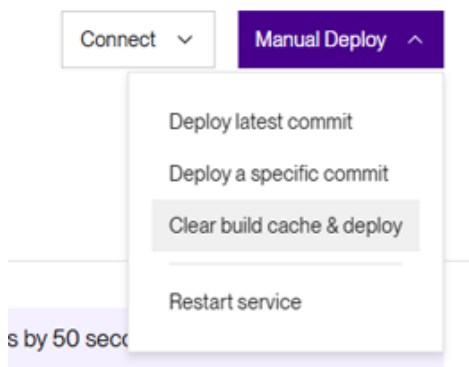


Figure 15.23: Clear build cache & deploy

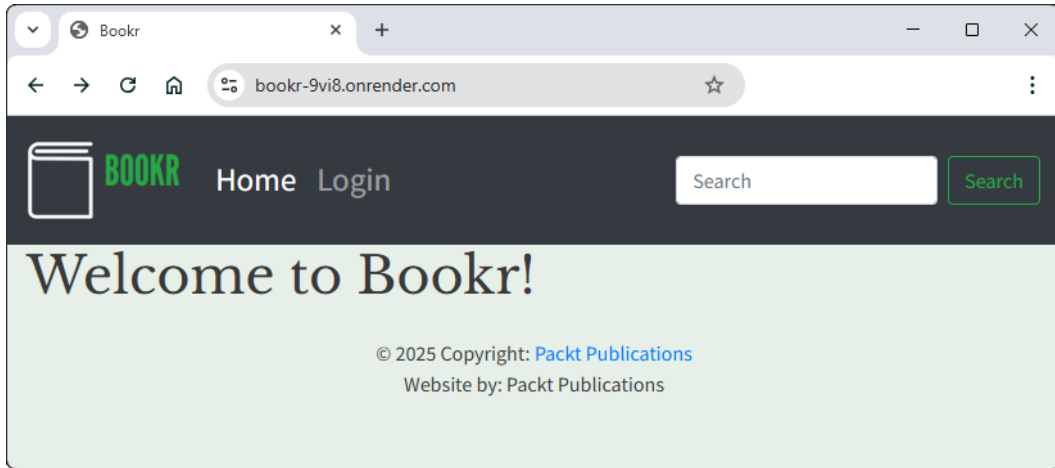


그림 15.22: 배포된 프로젝트

빌드 화면에서 오류가 있는지 주의 깊게 살펴보세요. 구성 매개변수 또는 환경 변수를 잘못 입력했을 수 있습니다. 다시 배포하기 전에 수정해야 할 소스 코드에 오류가 있을 수도 있습니다. 이 시점에서 와일드카드 대신 할당된 도메인을 ALLOWED_HOSTS 환경 변수에 추가하고 싶습니다:

```
ALLOWED_HOSTS=<yourdomain>.onrender.com
```

idx_7e8dbe76 오류 또는 재구성 후 다시 빌드할 때 가장 안전한 방법은 수동 배포 메뉴에서 Clear build cache & deploy를 클릭하는 것입니다.

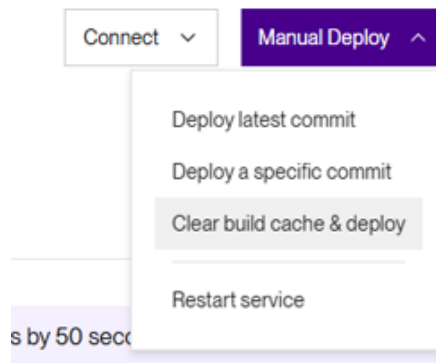


그림 15.23: 빌드 캐시 지우기 및 배포

If your deployment seems successful, confirm that the database has been migrated and the superuser created by logging in with the username and password details that you provided in *Step 16*.

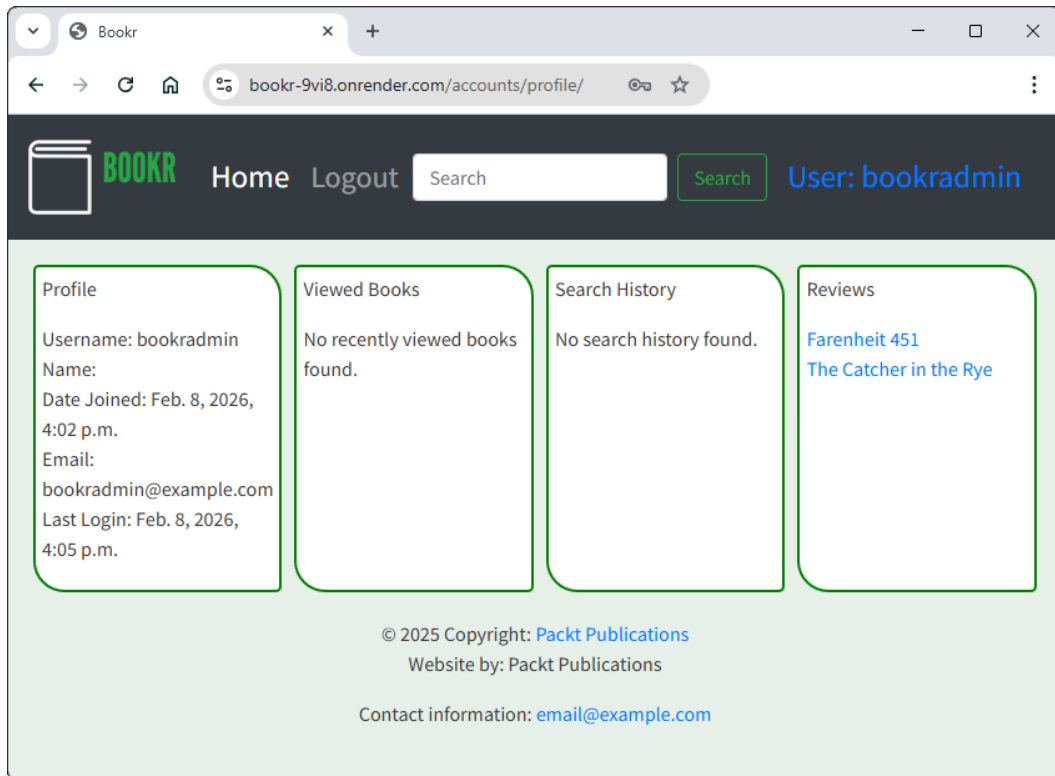


Figure 15.24: The Django project in production

With this, our production deployment is complete.

Summary

In this chapter, we have looked at simple architectures and deployment strategies for a Django project and learned the process of reconfiguring it from being a development instance to a production-ready deployment. Stepping through the deployment checklist, we have worked through the steps of a deployment to a PaaS provider, `render.com`.

Throughout this book, we have covered, in depth, everything required for setting up Python for installing Django: building an application to store data and accept data input through forms with validation, managing the system and the data through Django's admin system, and setting up a server to serve static files so that the application can be styled with CSS, enhanced

배포가 성공적인 것 같으면, 16단계에서 제공한 사용자 이름과 비밀번호로 로그인하여 데이터베이스가 마이그레이션되었고 슈퍼유저가 생성되었는지 확인하십시오.

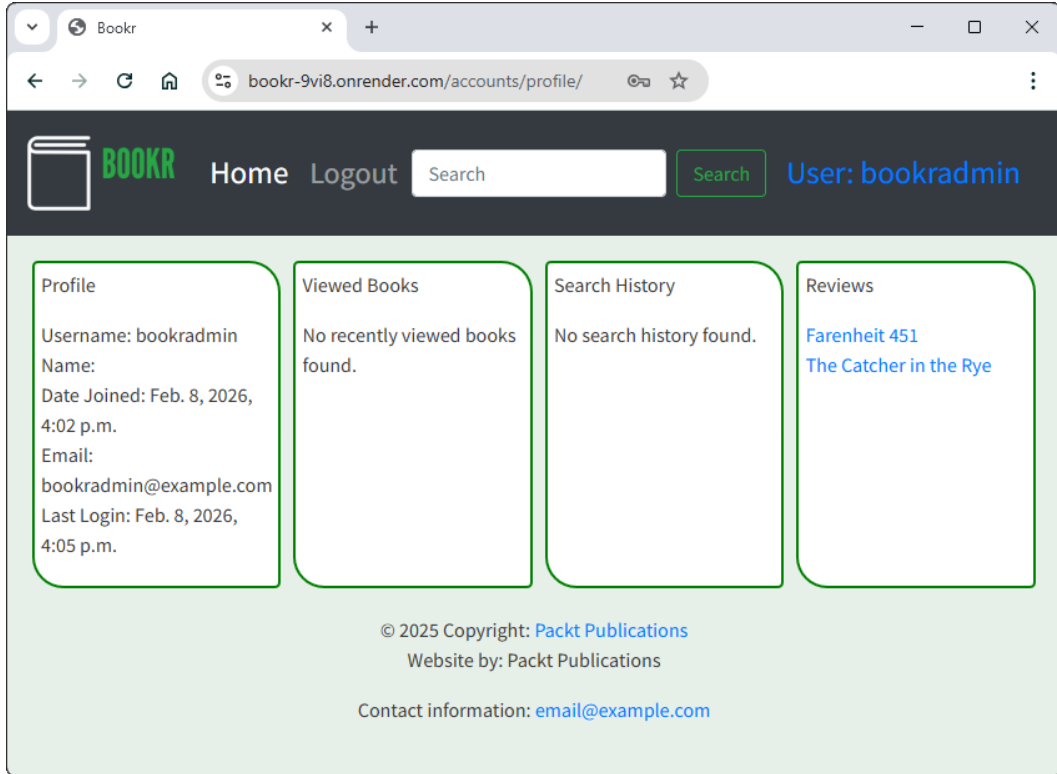


그림 15.24: 프로덕션에서의 Django 프로젝트

이로써 우리의 idx_31ab6dcb 프로덕션 배포가 완료되었습니다.

요약

이번 장에서는 Django 프로젝트를 위한 간단한 아키텍처와 배포 전략을 살펴보고, 개발 인스턴스에서 프로덕션 준비 완료 배포로 재구성하는 과정을 배웠습니다. 배포 체크리스트를 통해 PaaS 제공업체인 render.com에 배포하는 단계를 진행했습니다.

이 책 전반에 걸쳐, 우리는 Django 설치를 위한 Python 설정에 필요한 모든 것을 깊이 다루었습니다: 데이터를 저장하고 유효성 검사를 통해 양식으로 데이터 입력을 수락하는 애플리케이션 구축, Django의 관리 시스템을 통해 시스템과 데이터를 관리, 애플리케이션이 CSS로 스타일링될 수 있도록 정적 파일을 제공하는 서버 설정 등.

with JavaScript and display images. We have covered how to manage users and authentication, and in-depth customization to ensure Django is set up to better suit the project being undertaken. We also looked at how to provide data exports in formats such as CSV and PDF, concluding with how to make our application available to visitors by hosting the application on a server.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

JavaScript로 이미지를 표시합니다. 사용자 및 인증 관리 방법과 프로젝트에 더 적합하도록 장고를 설정하기 위한 심층 사용자 정의 방법을 다루었습니다. 또한 CSV 및 PDF와 같은 형식으로 데이터 내보내기를 제공하는 방법을 살펴보았으며, 애플리케이션을 서버에 호스팅하여 방문자가 애플리케이션을 사용할 수 있도록 하는 방법으로 마무리했습니다.

이 책의 PDF 사본, 코드 번들 등을 받으세요

QR 코드를 스캔하거나 packtpub.com/unlock으로 이동하세요. 이 책의 이름으로 검색하고, 판을 확인한 후 페이지의 단계를 따르세요.



UNLOCK NOW

참고: 송장 사본을 준비하세요. Packt 웹사이트에서 직접 구매한 경우 송장이 필요하지 않습니다.



packtpub.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packtpub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.



packtpub.com

7,000권 이상의 책과 비디오에 대한 전체 액세스를 위한 온라인 디지털 라이브러리에 가입하세요. 개인 개발 계획 및 경력 발전을 돕기 위한 업계 최고의 도구도 포함되어 있습니다. 자세한 내용은 저희 웹사이트를 방문해 주세요.

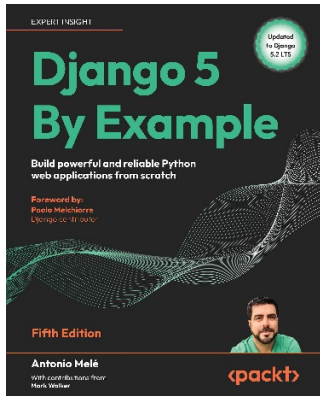
왜 구독해야 할까요?

- 4,000명 이상의 업계 전문가들이 제공하는 실용적인 전자책과 비디오로 학습 시간을 줄이고 코딩에 더 많은 시간을 할애하세요.
- 당신을 위해 특별히 제작된 스킬 플랜으로 학습을 향상시키세요.
- 매달 무료 전자책 또는 비디오를 받으세요.
- 중요한 정보에 쉽게 접근할 수 있도록 완전 검색 가능 • 콘텐츠를 복사하고 붙여넣기, 인쇄 및 북마크할 수 있습니다.

www.packtpub.com에서 무료 기술 기사를 모아 읽고, 다양한 무료 뉴스레터에 가입하며, Packt 도서 및 전자책에 대한 독점 할인 및 제안을 받을 수 있습니다.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



Django 5 By Example

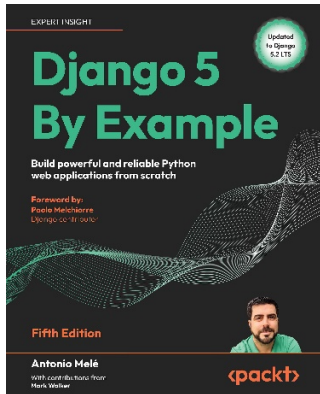
Antonio Melé

ISBN: 9781805125457

- Use various Django modules to solve specific problems using the latest features
- Integrate third-party Django applications into your project
- Build complex web applications using Redis, Postgres, Celery/RabbitMQ and Memcached
- Set up a production environment for your project with Docker Compose
- Build a RESTful API with Django Rest Framework (DRF)

당신이 즐길 수 있는 다른 책들

이 책을 즐기셨다면, Packt의 다른 책들에 관심이 있을 수 있습니다:

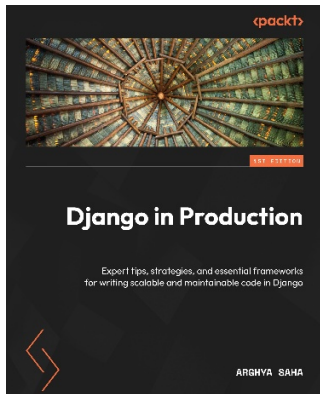


Django 5 예제로 배우기

안토니오 멜레

ISBN: 9781805125457

- 최신 기능을 사용하여 특정 문제를 해결하기 위해 다양한 Django 모듈을
- 사용하고 타사 Django 애플리케이션을 프로젝트에 통합하세요
- Redis, Postgres, Celery/RabbitMQ 및 Memcached를 사용하여 복잡한 웹 애플리케이션을 구축하세요
- Docker Compose로 프로젝트의 프로덕션 환경을 설정하세요
- Django Rest Framework (DRF)로 RESTful API를 구축하세요

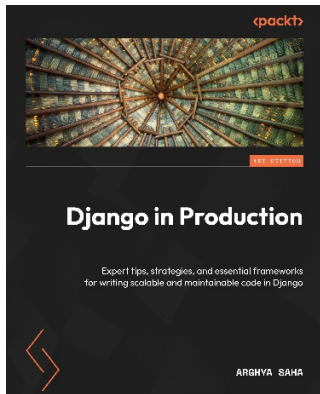


Django in Production

Arghya Saha

ISBN: 9781804610480

- Write scalable and maintainable code like a Django expert
- Become proficient in Docker for Django and experience platform-agnostic development
- Explore intelligent practices for continuous integration
- Leverage the power of AWS to seamlessly deploy your application in a production environment
- Optimize unstable systems through effective performance monitoring
- Effortlessly handle authentication and authorization issues
- Automate repetitive tasks by creating custom middleware
- Thoroughly test your code using factory_boy and craft comprehensive API tests



프로덕션에서의 장고

아르그야 사하

ISBN: 9781804610480

- 장고 전문가처럼 확장 가능하고 유지 관리가 용이한 코드를 작성하세요
- Django를 위한 Docker에 능숙해지고 플랫폼에 구애받지 않는 개발 경험하기
- 지속적인 통합을 위한 지능적인 관행 탐색하기
- AWS의 힘을 활용하여 프로덕션 환경에서 애플리케이션을 원활하게 배포하기
- 효과적인 성능 모니터링을 통해 불안정한 시스템 최적화하기
- 증 및 권한 부여 문제를 손쉽게 처리하기
- 커스텀 미들웨어를 생성하여 반복 작업 자동화하기
- factory_boy를 사용하여 코드를 철저히 테스트하고 포괄적인 API 테스트 작성하기

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packt.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Web development with Django 6*, we'd love to hear your thoughts! Scan the QR code below to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.



<https://packt.link/r/1836202075>

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Packt는 당신과 같은 저자를 찾고 있습니다

Packt의 저자가 되는 데 관심이 있으시다면, authors.packt.com을 방문하여 오늘 지원해주세요. 우리는 당신과 같은 수천 명의 개발자 및 기술 전문가와 협력하여 그들이 글로벌 기술 커뮤니티와 통찰력을 공유할 수 있도록 도왔습니다. 일반 지원서를 제출하거나, 저자를 모집 중인 특정 핫 주제에 지원하거나, 자신의 아이디어를 제출할 수 있습니다.

당신의 생각을 공유하세요

이제 Django 6로 웹 개발을 마쳤으니, 당신의 생각을 듣고 싶습니다! 아래의 QR 코드를 스캔하여 이 책의 아마존 리뷰 페이지로 바로 가서 피드백을 공유하거나 구매한 사이트에 리뷰를 남겨주세요.



<https://packt.link/r/1836202075>

당신의 리뷰는 우리와 기술 커뮤니티에 중요하며, 우리가 우수한 품질의 콘텐츠를 제공하고 있는지 확인하는 데 도움이 될 것입니다.

Index

A

ALLOWED_HOSTS 682, 683

API endpoint
creating, for top contributors page 587

API view
creating 579 – 582

AbstractUser
additional fields, adding to User class 492, 493
auth user model, replacing steps 493
password, modifying 493
subclassing 490
Username key, replacing 493

Admin app login template
repurposing 453 – 456

AdminSite
customizing 186 – 190
examining, from Python shell 181, 182
subclassing 182 – 186

AppDirectoriesFinder 217

Argon2 457

Asynchronous Server Gateway Interface (ASGI) 679

admin
automating, with Django Tasks 518 – 520

admin app 148
deletion behavior 177, 178
foreign keys 177, 178
groups, managing 160 – 165
models, registering 165
users, managing 160 – 165

admin interface
customizing 179

admin site
custom views, adding 511 – 515
custom views, restricting 511
customizing 488
views, adding 508

all() method

used, for retrieving set of objects 99, 100

app directories
HTML templates, finding 47

application programming interface (API) 531, 573, 656

assertions 646
types 649, 650
utilizing 646, 647

authentication
implementing 592
token-based authentication 593
views, testing 661

authentication app 443
templates, implementing 447 – 452
views, implementing 447 – 452

authentication data
used, for enhancing templates 466

authentication decorators 459 – 462
adding, to views 463 – 465

authentication models 444

authentication redirection 459 – 462

authentication schemes
reference link 593

authentication-based content
conditional blocks, using in templates 468, 469

automated test cases 644

automation testing 644

B

BCrypt/SHA256 457

Book Search page
session storage, using 484, 485

Book Search view
implementing, with Django Form 322 – 325

Book project's database
populating 117

Bookr 1
models and views, testing 673, 674

색인

A

ALLOWED_HOSTS	682, 683
API 엔드포인트	
상위 기여자 페이지 생성을 위한	587
API 보기	
생성 중	579–582
AbstractUser	
추가 필드, Userclass에 추가 하기	492, 493
인증 사용자 모델, 비밀번호 교체 단계, 서브클래스 수정	493
사용자 이름 키, 교체	490, 493
관리자 앱 로그인 템플릿 재사용	453–456
AdminSite	
사용자 정의 검토, Python 셸 서브클래스에서	186–190, 181, 182, 182–186
AppDirectoriesFinder	217
Argon2	457
비동기 서버 게이트웨이 인터페이스 (ASGI)	679
관리자	
Django 작업으로 자동화하기	518–520
관리자 앱	148
삭제 동작 외래 키 그룹 관리 모델 등록 사용자 관리	177, 178, 177, 178, 160–165, 165
	160–
관리자 인터페이스	
사용자 정의	165, 179
관리자 사이트	
사용자 정의 보기, 추가	511–
사용자 정의 보기, 제한	515, 511
사용자 정의 보기, 추가	488, 508
all() 메서드	

객체 집합을 검색하는 데 사용됨	99,
앱 디렉토리	1
HTML 템플릿, 찾기	0
응용 프로그램 프로그래밍 인터페이스 (API)	531, 573, 636, 4
주장	646
유형 활용	649, 650, 646, 647
인증	
구현	592
토큰 기반 인증 뷰, 테스트	593, 661
인증 앱	443
템플릿, 구현 뷰, 구현	447–452, 447–452
인증 데이터	
템플릿 향상을 위한 사용	466
인증 데코레이터	459–462
뷰에 추가	463–465
인증 모델 인증 리디렉션 인증 스킴	444, 459–462
참조 링크	593
인증 기반 콘텐츠 조건부 블록, 템플릿 사용	468, 469
자동화 테스트 케이스 자동화 테스트	644, 644
B	
BCrypt / SHA256도	457
서 검색 페이지	
세션 저장소, 사용하여	484, 485, 322
도서 검색 보기	–
구현하기, Django Form과 함께	325, 117
책 프로젝트의 데이터베이스 채우기	
북마크	1
모델 및 뷰, 테스트	673, 674

Bookr APIs			
used, for implementing token-based authentication	593 – 599		
Bookr project			
deploying	692		
Bookr project for deployment			
configuring	686 – 690		
Bootstrap	139		
for template styling	139, 140		
reference link	141		
base template			
creating	47 – 49		
login links, toggling	466, 467		
logout links, toggling	466, 467		
basic authentication	593		
binary file formats			
for data exports	607		
book catalog			
creating, with CBV	559 – 564		
book change page	173 – 177		
book cover			
displaying	438, 439		
book details view			
implementing	144		
book list item			
rendering, with template partials	556		
book search scaffold	63, 64		
bookr database			
entry, creating	90, 91		
browsers' sent values			
security and trust	390 – 392		
bubble plot	631		
generating	631 – 633		
built-in search			
using, to build custom admin dashboard	521, 522		
bulk_create() method	112		
used, for creating multiple records	112		
bulk_update() method	112		
used, for updating multiple records	113		
C			
CRUD operations, performing with CBVs	565		
Create view	565, 566		
Delete view	569, 570		
Read view	566		
Update view	567, 568		
CSS enhancements	260 – 262		
CSV files			
review summary CSV file, creating	604 – 606		
with Python csv module			
working with	602		
writing, with Python csv module	602 – 604		
Cascading Style Sheets (CSS)	139, 213		
Change publisher page	168 – 172		
Content Delivery Network (CDN)	225, 376		
Create operation	152 – 154		
user account, creating	154		
Create, Read, Update, and Delete (CRUD) operations	89, 147, 152		
Cross-Site Request Forgery Protection			
used, for form security	280		
cache	683		
cache invalidation			
file serving	248 – 250		
cached sessions	471		
change lists	167, 168		
class-based API views			
creating	584 – 586		
class-based views (CBVs)	120, 121, 531, 557, 558, 582		
in-built views	558		
testing	669		
used, for creating book catalog	559 – 564		
used, for performing CRUD operations	565		
clean method	332, 333		
code			
debugging	58 – 62		
simplifying, with ViewSets	588		
collectstatic management command			
static file, collecting for production	238, 239		
using	236 – 238		
comments	133		
common template variables			
accessing	510		
complex lookups			
performing, with Q objects	114		
complex query			
performing, with Q objects	114		
conditional blocks			

Bookr API		읽기 보기 업	566
토큰 기반 인증을 구현하는 데 사용됨	593 – 599	데이트 보기	567, 568
Bookr 프로젝트 배포	69	CSS 개선	260 – 262
배포를 위한 Bookr 프로젝트 구성	2686 – 690	CSV 파일	
부트스트랩	139	검토 요약 CSV 파일, Python csv 모듈로 생성하기	604 – 606
템플릿 스타일링	139,	작업 중	6026
참조 링크	140141	쓰기, Python csv 모듈로	02 – 604
기본 템플릿 생성하기	47 – 49466,	계단식 스타일 시트 (CSS) 발행자	139,
로그인 링크, 로그아웃 링크 전환, 전환	467466,	페이지 변경	213168 – 172
기본 인증 이전 파일 형식	593	콘텐츠 전송 네트워크 (CDN)	225, 376
데이터 내보내기용	60	생성 작업	152 – 154
도서 목록 생성하기, CBV로	7559 – 564	사용자 계정, 생성	154
책 변경 페이지책 표시	173 – 177	생성, 읽기, 업데이트 및 삭제 (CRUD) 작업	89, 147, 152
표시 중	438,	교차 사이트 요청 위조 방지 형식 보안을 위한 사용	280
책 세부 정보 보기 구현하기	4	캐시	683
책 목록 항목 렌더링, 템플릿 부분으로	3	캐시 무효화	
책 검색 스케폴드북r 데이터베이스	9	파일 제공	248 – 250
항목, 생성	63, 14	캐시된 세션 변경 목록	471
브라우저가 보낸 값 보안 및 신뢰	4	클래스 기반 API 뷰 생성	167, 168
버블 플롯 생성 중	904	클래스 기반 뷰 (CBVs)	
내장 검색 사용하여, 사용자 정의 관리 대시보드 구축	91390 – 392	내장된 뷰	558
bulk_create() 메서드	5	테스트	669
여러 레코드를 생성하는 데 사용됨	631	도서 카탈로그 생성을 위해 사용됨	559 – 564
bulk_update() 메서드	633	CRUD 작업을 수행하는 데 사용됨	565
여러 레코드를 업데이트하는 데 사용됨	112	clean	332, 333
C		methodcode	
CRUD 작업, CBV로 수행하기	565	디버깅	58 – 62
생성 보기	565,	ViewSet로 단순화하기	58
삭제 보기	566569,	collectstatic 관리 명령	8
	570	정적 파일, 프로덕션을 위한 수집	238, 239236 – 238
		댓글	133
		공통 템플릿 변수	
		510 접근하기	
		복잡한 조회	
		Q 객체	114로 수행하기
		복잡한 쿼리	
		Q 객체	114로 수행하기
		조건부 블록	

using, for authentication-based content in templates	468, 469	reference link	471
containerized deployment	691	Django Forms library	289, 290
context	52	building and rendering	307 – 311
context processor	381 – 383	form, defining	290 – 301
cookie-based sessions	471	form, rendering in template	301 – 307
create() method		used, for implementing Book Search view	322 – 325
used, for creating entry	92	Django ORM	68
cross-site request forgery (CSRF)	445	migration	72
custom admin dashboard		Django REST Framework (DRF)	573 – 575
building, with built-in search	521, 522	configuring	575
custom clean and validation methods		functional API views	576
implementing	338 – 345	installing	575
custom field validation	330	Django Storages	
custom filter function		reference link	255
implementing	535	Django Tasks	
custom filters		admin, automating	518 – 520
using, inside templates	536	periodic task exercise	523 – 527
custom inclusion tag		Django app	7, 25, 26
building	549 – 552	creating	7 – 9, 68 – 70
custom simple tag		Django application, deployment checklist	679
creating	544 – 546	environment-specific settings	682
template context, passing	547	manage.py check, running	680, 681
custom storage engines	255 – 257	manage.py runserver, switching	681
custom template filter		settings	681
creating	537 – 540	Django application, deployment checklist settings	
custom template filters	534	DEBUG	682
creating	534	SECRET_KEY	682
custom validators	331, 332	Django application, environment-specific settings	
custom view		ALLOWED_HOSTS	682, 683
adding, to admin site	511 – 515	cache	683
restricting, to admin site	511	databases	683
URLs, mapping	510, 511	default error views, customizing	685
D		EMAIL_BACKEND	683
DB Browser for SQLite	71	error reporting	685
reference link	71	HTTPS	684
DEBUG	682	performance optimizations	684
Delete operation	158, 159	Django development server	6, 24
Django	1	starting	7 – 9
used, for serving uploaded (and other) files	408 – 410	Django development server architecture	677, 678
Django Cookie Consent		Gunicorn	679
reference link	471	Whitenoise	679
Django Cookie Law		Django forms	
		initial values, adding	346 – 348

인증 기반 콘텐츠를 템플릿에서 사용하기 위해	468, 469	참조 링크 471	
컨테이너화된 배포	691	장고 폼 라이브러리 289, 290	
맥락	52	구성 및 렌더링 양식, 정의	307 – 311290 – 301301 – 307
맥락 프로세서 쿠키 기반 세션 create() 메서드	381 – 383471	양식, 템플릿에서 렌더링 BookSearch 뷰를 구현하는 데 사용됨	307 322 – 325
사용됨, 항목 92를 생성하기 위해		Django ORM 68	
교차 사이트 요청 위조 (CSRF) 사용자 정의 관리자 대시보드	445	마이그레이션 72	
내장 검색이 있는 빌딩 521, 522		장고 REST 프레임워크 (DRF) 573 – 575	
사용자 정의 클린 및 검증 방법 구현 338 – 345		구성	575
사용자 정의 필드 검증 사용	330	기능적 API 뷰 설치 중	576 575
자 정의 필터 함수 535 구현		장고 스토리지 참조 링크 255	
사용자 정의 필터 템플릿 내에서 사용 536		장고 작업 관리, 자동화 주기적 작업 연습	518 – 520523 – 527
사용자 정의 포함 태그 빌드 549 – 552		장고 앱 7, 25, 26 생성하기 7 – 9, 68 – 70	
사용자 정의 간단 태그 생성 중	544 –	장고 애플리케이션, 배포 체크리스트	679
템플릿 컨텍스트, 전달	54	환경별 설정 manage.py check, 실행 manage.py runserver, 전환 설정	682 680, 68 681 81
사용자 정의 저장 엔진 사용자 정의 템플릿 필터	255 – 25747	장고 애플리케이션, 배포 체크리스트 설정	
생성 537 – 540		DEBUG	682
사용자 정의 템플릿 필터 534 생성하기 534		SECRET_KEY	682
사용자 정의 검증기 사용	331, 332	Django 애플리케이션, 환경별 설정	
용자 정의 보기 추가, 관리자 사이트 제한, 관리자 사이트 URL 매핑	511 – 515511 510, 511	ALLOWED_HOSTS	682,
D		TS 캐시 데이터베이스	68 683 685
SQLite 71용 DB 브라우저 참조 링크 71		기본 오류 보기, 사용자 정의 EMAIL_BACKEND	683
DEBUG	682	오류 보고	685
삭제 작업 Django	158,	HTTPS	684
업로드된 (및 기타) 파일을 제공하는 데 사용됨	1591 408 – 410	성능 최적화	684
장고 쿠키 동의 참조 링크	471	장고 개발 서버 6, 24 시작 7 – 9	
장고 쿠키 법		장고 개발 서버 아키텍처	677, 678
		건이콘 화이트노이즈	679 67
		장고 폼 초기 값, 추가하기	9346 – 348

placeholders, adding	346 – 348
using, for file uploads	395 – 401
using, for image uploads	402 – 408
Django migration	70, 71
Django models	
creating	73, 74, 350, 351
editing	350, 351
field options	74 – 77
methods, implementing	85 – 87
ModelForm class	352 – 356
primary keys	77 – 79
publisher form, integrating	363 – 365
publisher form, styling	363 – 365
publisher, creating	357 – 362
publisher, editing	357 – 362
review creation UI	366 – 372
Review model	84, 85
testing	652 – 656
Django project	5, 6
creating	7 – 9
structure	22, 23
Django settings	44, 45
referring to, in code	45, 46
Django, deployment options	690
Bookr project, deploying	692 – 710
containerized deployment	691
manual deployment	690
Platform-as-a-service (PaaS)	691
data	
storing, in sessions	478
data exports	
binary file formats	607
database CRUD operations, Django	90
create() method, used for entry creation	92
entry, creating in bookr database	90, 91
object, creating with foreign key	92
database configuration	68 – 70
databases	683
debugging	57, 58
decoupled architecture	574
delete() method	
using	111
deletion behavior	
in admin app	177, 178
directory	
setting up, to store template filters	534, 535
double-underscore lookup	101

E

EMAIL_BACKEND	683
media files	684
static files	683
Elasticsearch	205
email server	
configuring	516, 517
error reporting	
ADMINS	685
logs	685
MANAGERS	685
errors	
debugging	53
handling	53
exceptions	53
generating	55 – 57
ImportError	54
IndentationError	53
IndexError	54
KeyError	54
NameError	54
SyntaxError	53
TypeError	54
viewing	55 – 57
exclude() method	
used, for retrieving objects	103

F

FieldFile	
custom storage engines	415
existing files, storing on	416 – 418
reading from	415, 416
working with	414
FileField	
model, creating with	421 – 426
FileSystemFinder	232, 233
field lookups	
used, for filtering	101, 102
used, for querying across many-to-many relationship	108, 109
fields	70
file	
downloading	392 – 395
storing, on model instances	410 – 413
uploading	392 – 395

파일 업로드를 위한 자
리 표시자, 이미지 업로
드를 위한 자리 표시자 346 –
348395 –
401402 –
408

Django 마이그레이
션 Django 모델 70, 71

생성하기 73, 74, 350,
351350,
35174 – 77
필드 옵션 85 –
메서드, 구현하기 87352 –
ModelForm 클래스기 35677 –
본 키 79363 –
출판사 양식, 통합출판사 365363 –
양식, 스타일링출판사, 365357 –
생성출판사, 편집 362357 –
362366 –
리뷰 생성 URI 37284,
뷰 모델 85652 –
테스트 656

장고 프로젝트 5, 6
구조 생 7 –
성 922, 23

장고 설정 44, 45
코드 45, 46에서 언급됨

Django, 배포 옵션 690
Bookr 프로젝트, 컨테이 692 –
너화된 배포 배포 71
수동 배포 690
서비스로서의 플랫폼 (PaaS) 691

데이터
세션에 저장 478

데이터 내보내기
이진 파일 형식 607

데이터베이스 CRUD 작업, Django 90
create() 메서드, 항목 생성을 위한 92
entry, bookr 데이터베이스 객체에서 90,
생성, 외래 키로 생성 9
데이터베이스 구성 68 1
9
databases 70683
2
디버깅 57,
58574

분리된 아키텍처 delete() 58574

메서드
111 사용 중

삭제 동작
관리 앱에서 177, 178

디렉토리
설정하기, 템플릿 필터 저장하기 534, 535

더블 언더스코어 조회 101

E

EMAIL_BACKEND 683
미디어 파일 684
정적 파일 683

엘라스틱서치 205

이메일 서버
구성 516, 517

오류 보고
관리자 685
로그 685
매니저 685

오류
디버깅 53
처리 53

예외 53
생성 55 – 57
ImportError 54
IndentationError 53
IndexError 54
KeyError 54
NameError 54
SyntaxError 53
TypeError 54
보기 55 – 57

exclude() 메서드
객체 103을 검색하는 데 사용됨

F

FieldFile
사용자 정의 저장 엔진 415
기존 파일, 저장하기 416 – 418
읽기 415, 416에서
작업하기 414와

파일 필드
모델, 생성하기 421 – 426

FileSystemFinder 232, 233

필드 조회
사용됨, 필터링을 위해 101, 102
사용됨, 다대다 관계를 쿼리하기 108, 109
위해

fields 70

file
다운로드 중 392 –
저장, 모델 인스턴스에 업 395410 –
로드 중 413392 –
395

file saving			
handling		432, 433	
file serving			
for cache invalidation		248 – 250	
file uploads			
Django forms, using for		395 – 401	
HTML forms, using for		386 – 388	
ModelForm, using for		426 – 428	
file-based sessions		471	
filter() method			
using, for object retrieval		100, 101	
filtering			
field lookups, using		101, 102	
filters		133, 531	
findstatic command		243	
used, for finding files		244 – 247	
flexbox		481	
foreign keys			
in admin app		177, 178	
used, for creating object		92	
used, for querying		107	
form		268	
building, in HTML		273 – 280	
built-in field validation		318, 319	
data, accessing in View		280	
defining		290 – 301	
element		268, 269	
extra field validation, adding		319 – 322	
query string, need for		288, 289	
rendering, in template		301 – 307	
selecting, between GET or POST request		286 – 288	
validating		311 – 315	
validating, in view		315 – 318	
working, with POST data in view		281 – 285	
form enhancements, ModelAdmin attributes			
filter_horizontal / filter_vertical		498, 499	
get_readonly_fields		503	
inline forms		499 – 502	
list_editable		504	
radio_fields		503	
readonly_fields		503	
function-based views (FBVs)		120, 121, 557	
implementing		124 – 126	
functional API views		576	
functional testing		645	
G			
GET request			
working with		39 – 42	
GET values		42, 43	
GET, or POST request			
selecting between		286 – 288	
Gunicorn		679	
generic views		582	
get() method			
used, for returning object		98, 99	
using, to retrieve object		97, 98	
global logo			
adding		262, 263	
graphs		628	
bubble plot		631 – 633	
generating, in Python		633 – 636	
plot, generating		630	
groups			
managing		159	
managing, through admin app		160 – 165	
H			
HTML			
form, building		273, 274, 278 – 280	
HTML forms			
using, for file uploads		386 – 388	
HTML templates			
finding, in app directories		47	
HTTP 403 Forbidden response		409	
HTTP headers		16 – 18	
HTTP methods		16	
HTTP request		15	
HTTP response		15	
HTTP status codes		19	
reference link		19	
HTTPS		684	
CSRF_COOKIE_SECURE		684	
SESSION_COOKIE_SECURE		684	
Help Desk user group		160	
HyperText Markup Language (HTML)		12, 213	
Hypertext Transfer Protocol (HTTP)		15, 573	

파일 저장		
처리	432, 433	
파일 제공		
캐시 무효화	248–250을 위한	
파일 업로드		
Django 폼, HTML 폼	395 –	
사용, ModelForm 사용	401386 –	
용, 사용을 위한	388426 –	
	428	
파일 기반 세션filter()	471	
메서드		
객체 검색을 위한 사용	100, 101	
필터링		
필드 조회, 101, 102 사용		
필터	133,	
findstatic 명령	53	
파일 찾기에 사용됨	244–247	
flexbox 외래	481	
키		
관리 앱에서	177,	
객체 생성을 위한 사용	1	
쿼리를 위한 사용	177	
	8	
양식 268		
HTML에서의 빌딩	273,	
내장 필드 유효성 검	280318,	
사 데이터, 보기 정의	319280	
에서 접근	290–	
요소	301268,	
추가 필드 검증, 쿼리 문자	269319–	
열 추가, 렌더링 필요, 템플	322288,	
릿에서	289301–	
선택, GET 또는 POST 사이에서	307	
요청 검증, 뷰에	286–288	
서 검증	311 –	
	315315 –	
	318281 –	
	285	
작업, 뷰에서 POST 데이터와 함께		
양식 개선, ModelAdmin속성		
filter_horizontal / filter_vertical	498,	
get_readonly_fiel	499503	
dsinline	499 –	
formlist_editable	502504	
radio_fieldsreado	503503	
nly_fields		
함수 기반 뷰(FBVs)	120, 121, 557	
구현	124–126	
함수형 API 뷰 576		
기능 테스트 645		
G		
GET 요청		
작업하기	39–42	
GET 값	42, 43	
GET 또는 POST 요청		
선택하기	286–288	
Gunicorn	679	
제네릭 뷰 get()	582	
메서드		
객체를 반환하기 위한 사용,	98,	
객체를 검색하기 위해 사용	9997,	
	98	
글로벌 로고		
262, 263 추가		
그래프 628		
버블 플롯 생성, 파	631 –	
이썬 플롯에서 생성	633633–	
	636630	
그룹		
관리	1591	
관리, 관리자 앱을 통해	60–165	
H		
HTML		
양식, 빌딩	273, 274, 278–280	
HTML 양식		
파일 업로드를 위한 사용	386–388	
HTML 템플릿		
앱 디렉토리에서 찾기	47	
HTTP 403 금지 응답HTTP 헤더	409	
	16 –	
HTTP 메서드	1816	
HTTP 요청	15	
HTTP 응답	15	
HTTP 상태 코드	19	
참조 링크	19	
HTTPS 684		
CSRF_COOKIE_SECURE	684	
SESSION_COOKIE_SECURE	684	
헬프 데스크 사용자 그룹하이퍼텍스	160	
트 마크업 언어(HTML)	12, 213	
하이퍼텍스트 전송 프로토콜	15, 573	
(HTTP)		

I

ImageField	
model, creating with	421 – 426
PIL images, writing to	418 – 420
i18n module	452
image and PDF, books	
uploading	433 – 438
image uploads	
Django forms, using for	402 – 408
ModelForm, using for	426 – 428
images	
generating in Python, with Pillow	636 – 638
resizing, with Pillow	404
storing, on model instances	413, 414
inclusion tags	542, 548
implementing	548
used, for rendering details on user profile page	552, 553
using, inside template	549
initial values	
adding, to Django form	346 – 348
adding, to OrderForm class	348 – 350
integration testing	645

J

JavaScript Object Notation (JSON)	472, 473, 578
Juggler	116

L

Linux	
pyenv, installing on	4
LiveServerTestCase class	671
list of books and reviews	
displaying	134 – 136
login links	
toggling, in base template	466, 467
logout links	
toggling in base template	466, 467

M

MEDIA_URL	
using, in templates	381 – 386
ManifestFilesStorage storage engine	

exploring	251 – 254
Model View Controller (MVC)	10
Model-View-Template (MVT)	10
examples	12 – 14
ModelAdmin classes	
customizing	190, 191, 209 – 211
date list_filter, adding	200 – 202
date_hierarchy filters, adding	200 – 202
display decorator	197, 198
fields, excluding and grouping	206 – 209
filter	198 – 200
list display fields	191 – 197
search bar	203 – 206
ModelForm	
using, for file and image uploads	429 – 432
ModelForm class	352 – 356
macOS	
pyenv, installing on	4
manage.py	
commands	23
manual deployment	690
many-to-many query	
objects, using	109
set() method, using	109
many-to-many relationships	81 – 83
add() method, using	95, 96
create() method, using	96
querying, with field lookup	108
records, creating with	94, 95
set() method, using	96, 97
many-to-one relationship	80, 81
records, creating	93, 94
media files	375
media files, in development	
serving	376, 377
media serving	
configuring	377 – 381
settings	376
media storage	
configuring	377 – 381
media uploads	
settings	376
middleware	444, 445
middleware modules	445, 446
purpose	445, 446
middleware stack	444

I

이미지 필드	
모델, PIL 이미지로 생성, 쓰기	421–426 418–420
i18n 모듈	452
이미지 및 PDF, 책 업로드 중	433–438
이미지 업로드	
장고 폼, ModelForm 사용, 사용하기 위해	402–408 426–428
이미지	
파이썬에서 생성, Pillow로 크기 조정, Pillow로 모델 인스턴스에 저장	636–638 404 413, 414
포함 태그	542, 548
구현하기	548
사용됨, 사용자 프로필 페이지의 세부정보 렌더링을 위해	552, 553
사용 중, 템플릿 내에서	54
초기 값	
Django 폼에 추가하기, OrderForm 클래스에 추가하기	346– 348 348–350
통합 테스트	645

J

자바스크립트 객체 표기법 (JSON)	472, 473, 578
Juggler	116

L

Linux	
pyenv, 4에 설치	
LiveServerTestCase 클래스	671
스의 책 및 리뷰 목록	134–136 표시
로그인 링크	
전환, 기본 템플릿 466, 467에서	
로그아웃 링크	
기본 템플릿에서 전환 466, 467	

M

MEDIA_URL	
템플릿에서 사용 381–386	
ManifestFilesStorage 저장 엔진	

탐색	251–254
모델 뷰 컨트롤러 (MVC)	10
모델-뷰-템플릿 (MVT)	10
예제 12–14	
ModelAdmin 클래스	
사용자 정의	190, 191, 209–211
날짜 list_filter,	200–
date_hierarchy 필터 추가,	202
display 데코레이터 추가	201
필드, 제외 및 그룹화	198
필터	209
목록 표시 필드	198
검색 바	201
모델 폼	
파일 및 이미지 업로드에 사용,	429–432
모델 폼 클래스	352–356
macOS	
pyenv, 4에 설치하기	
manage.py	
commands 23	
수동 배포 다대다 쿼리	690
객체, 사용하여	109
set() 메서드, 사용	109
다대다 관계 81–83	
add() 메서드, 사용	95,
create() 메서드, 사용	9
쿼리, 필드 조회 레코드	109
로, set() 메서드로 생성,	94, 95
사용	97
다대일 관계 80, 81	
레코드 생성 93, 94	
미디어 파일	375
미디어 파일, 개발 중	
제공 376, 377	
미디어 제공	
설정 구성	377–
	38
미디어 저장	13
구성 377–381	76
미디어 업로드	
설정 376	
미들웨어 미들웨어 모듈	444, 445
목적 445, 446	446
미들웨어 스택 444	

migration	70	order_by() method	
model instances		used, for retrieving objects	103 – 106
files, storing on	410 – 414		
images, storing on	413		
model methods	85		
model serializers	583, 584		
creating	584 – 586		
models	10		
registering, within admin app	165		
multi-field validation	334 – 337		
myproject directory	23, 24		
N			
navigation bar			
adding	141, 142		
non-field errors	336		
O			
Object Relational Mapper (ORM)	10		
OrderForm class			
initial values, adding	348 – 350		
placeholders, adding	348 – 350		
object			
creating, with foreign key	92		
object creation, with foreign key			
create() method, using for many-to-	96, 97		
many relationships			
many-to-many relationship, using	95, 96		
add() method			
records, creating for many-to-many	94, 95		
relationships			
records, creating for many-to-one	93		
relationship			
set() method, using for many-to-many	96, 97		
relationships			
object instance			
used, for querying across foreign key	107, 108		
relationships			
object retrieval			
exclude() method, using for	103		
filter() method, using for	100, 101		
order_by() method, using for	103 – 106		
one-to-one relationships	83		
operation filtering			
pattern matching, using	102		
		P	
		PBKDF2/SHA256	457
		PDF files	
		additional functionality, adding to	626 – 628
		view	
		working with	621, 622
		PDF version, of web pages	
		generating, in Python	622 – 625
		PIL images	
		writing, to ImageField	418 – 420
		POST data, in view	
		working with	281 – 285
		POST request	
		working with	39 – 42
		Pillow	402, 636
		image, resizing with	404
		Platform-as-a-service (PaaS)	691
		Portable Document Format (PDF)	621
		PyCharm	476
		project setup	26 – 34
		setup	26
		Python	
		graphs, generating	633 – 636
		images, generating with Pillow	636 – 638
		used, for writing to CSV files	603
		values, retrieving	311 – 315
		Python Imaging Library (PIL)	402
		Python shell	
		AdminSite object, examining from	181, 182
		PythonAnywhere.com	691
		Python's csv module	
		CSV files, writing with	602
		working with	602
		pagination customizations	506
		parallelization	
		using, to run tests	672, 673
		password storage, in Django	457
		pattern matching	
		used, for filtering operations	102
		performance optimizations	684
		CONN_MAX_AGE	684
		sessions	684

migrationmodel	70	order_by() 메서드 객체를 검색하는 데 사용됨	103-106
인스턴스		P	
파일, 이미지에 저장, 저장 중	410- 41	PBKDF2 / SHA256	457
모델 메서드 모델 직	44	PDF 파일	
렬 변환기	583, 584	추가 기능, toview 추가	626-628
생성 584-586			
모델 10		621, 622 작업하기	
등록, 관리자 앱 내 165		웹 페이지의 PDF 버전	
다중 필드 검증	334 -	생성, 파이썬 622-625	
myproject 디렉토리	33723 , 24	PIL 이미지	
N		작성, ImageField에 418-420	
탐색 모음		POST 데이터, 뷰에서	
141, 142 추가		작업하기 281-285	
비필드 오류 336		POST 요청	
		39-42 작업하기	
O		Pillow 402, 636	
객체 관계 매퍼 (ORM) OrderForm	10	이미지, 404로 크기 조정하기	
클래스		서비스로서의 플랫폼 (PaaS)	691
초기 값, 자리 표시	348- 350348- 350	휴대용 문서 형식 (PDF)	621
자 추가, 추가		PyCharm	476
객체		프로젝트 설	26-
생성, 외래 키로 92		정설정	3
객체 생성, 외래 키 사용		파이썬	4
create() 메서드, 다대다 관계를 위한 사용	96, 97	그래프, 생성	633- 636
다대다 관계, add() 메서드 사용	95, 96	이미지, CSV 파일에 쓰기 위 해 Pillow로 생성, 값 검색	636636- 638603
			311-315
기록, 다대다 관계를 위한 생성	94, 95	파이썬 이미지 라이브러리 (PIL) 파	402
레코드, 다대일 관계를 위한 생 성	93	이썬 셀	
set() 메서드, 다대다 관계를 위해 사 용	96, 97	AdminSite 객체, 181, 182에서 검사	
객체 인스턴스		PythonAnywhere.com	691
외래 키 관계를 통해 쿼리하는 데 사용	107, 108	Pythons csv 모듈	
객체 검색		CSV 파일, 작성하기	602
exclude() 메서드, forfilter	103100,	작업하기	602
() 메서드 사용, fororder_by	101103	pagination	506
() 메서드 사용, for	-106	customizationsparallelization	
일대일 관계 운영 필터링	83	using, to run tests 672, 673	
패턴 매칭, 사용하기	102	password storage, in	457
		Djangopattern matching	
		필터링 작업에 사용됨 102	
		성능 최적화 684	
		CONN_MAX_AGE	684
		세션	684

templates	685
pickle	472
placeholders	
adding, to Django form	346 – 348
adding, to OrderForm class	348 – 350
plot	630
generating	630
rendering, on web page	631
plotly	628
graphs, generating with	628
installation	629
plotly Figure object	
setting up	629
primary keys	77
profile page	457
adding	457 – 459
project management application	
models, creating	116
publisher	
creating	357 – 362
editing	357 – 362
publisher form	
integrating	363 – 365
styling	363 – 365
pyenv	
installing, on Linux	4
installing, on macOS	4
installing, on Windows	3
virtual environment, creating	3 – 5

Q

Q objects	
used, for performing complex lookups	114
used, for performing complex query	114
QueryDict object	42, 43
working with	39 – 42
querying	
across foreign key relationship, with	107, 108
object instance	
across many-to-many relationship, with	108
field lookup	
across relationships	106
foreign keys, using	107
many-to-many query, using objects	109
many-to-many query, using set() method	109
model name, using	107
update() method, using	110

queryset	
verifying, with contains method	115, 116

R

REST APIs	573, 574
creating	576 – 578
RESTful web service	575
Railway.com	691
Render.com	691, 700
RequestFactory	666
using, to test views	666 – 669
Retrieve operation	155, 156
Review model	
adding	84, 85
Reviews logo	
adding	258 – 260
Routers	588
URL configuration, using	588
using	589 – 594
read operations	
all() method, using to retrieve set of	99, 100
object	
get() method, using for returning	98, 99
object	
get() method, using to retrieve object	97, 98
performing, with get() method	97
regression testing	645
relationships, relational database	79
many to many	81 – 83
many to one	80, 81
one-to-one	83, 84
remote user authentication	593
render function	
templates, rendering	49
representational state transfer (REST)	574
request	
processing	20, 21
request.user object	457
reviewer profile model	
implementing	497
reviews app	
migrating	87 – 89
reviews model	
registering	166, 167

템플릿 685	
피클 플레이스	472
홀더	
장고 폼에 추가하기,	346 –
OrderForm 클래스에 추가하기	348348 – 350
플롯 630	
생성하기	630
웹 페이지에서 렌더링하기	631
plotly 628	
그래프, 생성하기	628
설치	629
plotly Figure 객체	
설정하기 629	
기본 키	77
프로필 페이지	457
추가 457 – 459	
프로젝트 관리 애플리케이션	
모델, 116 생성	
출판사	
생성편	357 –
집	362357 – 362
출판사 양식	
스타일 통	363 –
합	365363 – 365
pyenv	
리눅스에 설치	4
macOS에 설치	4
Windows 가상 환경에 설	3
치, 생성	3 – 5
Q	
Q 객체	
복잡한 조회를 수행하는 데 사용	114
복잡한 쿼리를 수행하는 데 사용	114
QueryDict 객체 42, 43	
39 – 42로 작업하기	
쿼리	
외래 키 관계를 통해, 객체 인스턴	107, 108
스와 함께	
다대다 관계를 통해, 필드 조회와 함	108
께	
관계 전반에 걸쳐	106
외래 키, 사용하여	107
다대다 쿼리, 객체 사용하여	109
다대다 쿼리, set() 메서드 사용	109
모델 이름, 사용 중	107
update() 메서드, 사용 중	110

queryset
확인 중, contains 메서드 115, 116

R

REST API 573, 574	
생성 576 – 578	
RESTful 웹 서비스	575
Railway.comRe	691
nder.comRequ	691,
estFactory	70
뷰 테스트 사용 666 – 669	06
검색 작업검토 모델	155, 156

84, 85 추가

리뷰 로고

258 – 260 추가

라우터 588

URL 구성, 사용 사용 5885
89 – 594

읽기 작업

all() 메서드, 객체 집합을 검색하	99, 100
는 데 사용	
get() 메서드, 객체를 반환하는	98, 99
데 사용	
get() 메서드를 사용하여 객체를	97,
검색하는 데 사용	9

회귀 테스트

관계, 관계형 데이터베이스

다대다 다대	81 –
일 일대일	8380,
	8183,
	84

원격 사용자 인증 렌더 함수

템플릿, 렌더링 49

표현 상태 전이(REST)

574

요청

처리 중 20, 21

request.user 객체리뷰어

457

프로필 모델

구현 중인 49

리뷰 앱

마이그레이션 중 787 – 89166,

리뷰 모델

등록 중인 167

S

SECRET_KEY	682
SQLite3	68
STATICFILES_DIRS prefixed mode	240 – 243
Seenode.com	691
SimpleTestCase class	670
features	670
Structured Query Language (SQL)	11
sample link	
displaying	439
search and filter customization	507
serializers	578
class-based views	582
generic views	582
session authentication	593
session engine	444, 470
cached sessions	471
cookie-based sessions	471
file-based sessions	471
session key	
examining	473 – 478
session storage	
using, for Book Search page	484, 485
sessions	470
cookie content, flagging	471
data, storing	478
recently viewed books, storing	479 – 483
simple tags	542
using, inside templates	544
simple template tag	
creating	542
directory, setting up	543
implementing	543
template library, setting up	543
singleton class	535
site welcome screen	
creating	63
site-wide Django admin	
AdminSite object, examining from	181, 182
Python shell	
AdminSite, subclassing	182 – 186
AdminSiteobject, customizing	186 – 190
customizing	179 – 181
smoke testing	645

soft delete	157
static URLs	
generating, with static template tag	224 – 226
static file finders	216
AppDirectoriesFinder	217
request	217
static file namespacing	218, 219
static file serving	215, 216
from app directory	220 – 224
from project static directory	233 – 236
static template tag	225, 226
example	226 – 229
using	229 – 232
storage engine	
global logo, adding	262
string filters	536, 537
superuser account	
creating	149 – 151

T

TitledUser model	
User, replacing with	494 – 496
TransactionTestCase class	
features	671
Transmission Control Protocol (TCP)	575
template context	
passing, in custom simple tag	547
template filters	532, 533
directory, setting up to store	534, 535
template inheritance	138, 139
adding	141, 142
template language	132
template library	
setting up	535
template partials	531, 554, 555
book list item, rendering	556
template styling	
with Bootstrap	139, 140
template tags	132, 531, 541
inclusion tags	542
simple tags	542
template variables	132
using, to pass additional keys to	515, 516
templates	
templates	12, 127 – 129

S

비밀_키	682
SQLite3 정적파일_디렉토리_접두	68
사모드	240 – 243
Seenode.com	691
간단한 테스트 케이스_클래스 기능 670	670
구조적 쿼리 언어 (SQL) 샘플 링크	11
표시 439	
검색 및 필터 사용자 정의	507
직렬 변환기	578
클래스 기반 뷰	582
일반 뷰	582
세션 인증 세션 엔진	593
	444, 470
캐시된 세션	471
쿠키 기반 세션	471
파일 기반 세션	471
세션 키	
검사 473–478	
세션 저장소	
사용 중, 도서 검색 페이지 484, 485	
세션 470	
쿠키 내용, 플래그 지정	471
데이터, 저장	4784
최근에 본 책, 저장	79–483
간단한 태그 542	
사용하기, 템플릿 안에서 544	
간단한 템플릿 태그	
생성하기	542
디렉토리, 설정하기	543
구현 중	543
템플릿 라이브러리, 설정 중	543
싱글톤 클래스	535
사이트 환영 화면	
63 생성	
사이트 전체 Django 관리자	
AdminSite 객체, Python 셸에서 검사	181, 182
AdminSite, 서브클래싱	182–
AdminSite 객체, 커스터마이	186186–
징커스터마이징	190179–
	181
스모크 테스트 645	

소프트 삭	157
제_정적_URL	
정적 템플릿 태그 224–226으로 생성하기	
정적 파일 찾기 216	
AppDirectoriesFinder	217
요청	217
정적 파일 네임스페이스 정적	218,
파일 제공	219215,
앱 디렉토리에서	216
프로젝트 정적 디렉토리에서	220–
	224233–
	236
정적 템플릿 태그 225, 226	
예제사	226–
응	229229–
	232
저장 엔진	
전역 로고, 추가 262	
문자열 필터슈퍼유저	536, 537
계정	
생성하기 149–151	

T

TitledUser 모델	
사용자, 494–496으로 교체	
TransactionTestCase 클래스	
기능 671	
전송 제어 프로토콜 (TCP) 템플릿 컨텍스트	575
사용자 정의 간단 태그 547에서 전달	
템플릿 필터 532, 533	
디렉토리, 저장을 위한 설정 534, 535	
템플릿 상속 138, 139	
추가 141, 142	
템플릿 언어 템플릿 라	132
이브러리	
설정 535	
템플릿 부분 531, 554, 555	
책 목록 항목, 렌더링 556	
템플릿 스타일링	
부트스트랩 139, 140과 함께	
템플릿 태그 132, 531, 541	
포함 태그	542
간단한 태그	542
템플릿 변수 132	
추가 키를 템플릿에 전달하기	515, 516
위해 사용	
템플릿	12, 127 – 129

- custom filters, using 536
- custom simple tag, using 549
- enhancing, with authentication data 466
- greeting message, displaying 129 – 131
- media, referring in 421
- rendering, in view 49, 50
- rendering, with render function 49
- simple tags, using 544
- variables, rendering 50, 51
- variables, using 51, 52
- templates directory**
 - creating 47 – 49
- test case classes** **670**
 - LiveServerTestCase class 671
 - SimpleTestCase class 670
 - TransactionTestCase class 671
- test cases** **646**
 - implementing 646
 - pre-test setup and cleanup, performing 650, 651
- test code**
 - modularizing 671
- testing** **643**
 - automation testing 644
 - functional testing 645
 - importance 644
 - in Django 645
 - integration testing 645
 - regression testing 645
 - smoke testing 645
 - unit testing 645
- tests**
 - running, with parallelization 672, 673
- token authentication** **593**
 - implementing, for Bookr APIs 593 – 599
- top contributors page**
 - used, for creating API endpoint 587
- U**
- URL configuration** **122, 123**
 - with Routers 588
 - with ViewSets 588
- URL mapping** **36, 37, 124**
 - parameters 288, 289
- URLs**
 - mapping, for custom view 510, 511
- Update operation** **156, 157**
- unit test**
 - writing 648, 649
 - writing, for views 657 – 660
- unit testing** **645**
 - implementing 646
- update() method**
 - using 110
- uploaded files**
 - working with, in view 388, 389
- user account**
 - creating 154
- user model** **488**
 - extending 489, 490
- user profile page**
 - details, rendering with inclusion tags 552, 553
- users**
 - managing 159
 - managing, through admin app 160 – 165
- V**
- ViewSets** **588**
 - URL configuration, using 588
 - used, for simplifying code 588
 - using 589 – 592
- variables**
 - rendering, in templates 50, 51
 - using, in templates 51, 52
- view function**
 - creating 509
- views** **11, 35, 36, 120, 556**
 - adding, to admin site 508
 - class-based views 120, 121, 557, 558
 - function-based views 120, 121, 557
 - template, rendering 49, 50
 - test cases, writing, to validate authenticated users 661 – 665
 - testing 656, 657
 - testing, with authentication 661
 - testing, with RequestFactory 666 – 669
 - unit tests, writing 657 – 660
 - URL map, setting up 38
 - writing 38
- virtual environment**
 - creating, with pyenv 3 – 5

사용하는 사용자 정의 필터	536		
사용하는 사용자 정의 간단 태그	549		
인증 데이터 인사말 메시지로 향상, 미디어 참조	466		
	129 – 131		
보기에서 렌더링	49,		
렌더 함수로 렌더링	50		
간단한 태그, 변수	544		
사용, 변수 렌더링, 사용	50, 515, 52		
템플릿 디렉토리			
47 – 49 생성			
테스트 케이스 클래스 670			
LiveServerTestCase 클래스	671		
SimpleTestCase 클래스	670		
TransactionTestCase 클래스	671		
테스트 케이스 646			
구현하기	646		
사전 테스트 설정 및 정리, 수행 중	650, 651		
테스트 코드			
모듈화	671		
테스트 643			
자동화 테스트	644		
기능 테스트	645		
중요성	644		
장고에서	645		
통합 테스트	645		
회귀 테스트	645		
스모크 테스트	645		
단위 테스트	645		
테스트			
실행 중, 병렬화	672, 673		
토론 인증 593			
구현 중, Bookr API를 위한	593 – 599		
상위 기여자 페이지			
사용됨, API 엔드포인트 생성을 위해	587		
U			
URL 구성 122, 123			
라우터와 함께	588		
ViewSets와 함께	588		
URL 매핑 36, 37, 124			
parameters	288, 289		
URLs			
mapping, for custom view	510, 511		
Update operation	156, 157		
단위 테스트			
뷰를 위한 작성		648, 649	657 – 660
단위 테스트 645			
구현하기	646		
update() 메서드			
사용하기	110		
업로드된 파일			
작업 중, 보기	388, 389		
사용자 계정			
생성	154		
사용자 모델 488			
확장	489, 490		
사용자 프로필 페이지			
세부사항, 포함 태그로 렌더링	552, 553		
사용자			
관리		1591	
관리, 관리 앱을 통해		60 – 165	
V			
ViewSets 588			
URL 구성, 사용 중		588	
사용됨, 코드 단순화를 위해 사용		588	89 – 592
변수			
렌더링, 템플릿에서		50, 515, 52	
사용, 템플릿에서			
뷰 함수			
생성	509		
views 11, 35, 36, 120, 556			
추가, 관리자 사이		508	
트에 클래스 기반 뷰		120, 121, 557,	
함수 기반 뷰 템플릿, 렌더링		558	120, 121, 557
test cases, writing, to validate authenticated users		557	49, 50
testing		661 – 665	
testing, with authentication		656,	
testing, with authentication		657	661
RequestFactory unit tests, writing		666 – 669	657
URL 맵, 설정하기			
작성하기		– 660	38
가상 환경			
pyenv 3 – 5로 생성하기			

W

WeasyPrint	621
Web Server Gateway Interface (WSGI)	679
Whitenoise	679
Windows	
pyenv, installing on	3

X

XLSX files	608
data, writing to workbook	611
data, writing to worksheet	610

formatted spreadsheet, developing	615 – 618
spreadsheet, formatting	611 – 614
workbook	608
workbook, creating	610
worksheet, creating	610
worksheets	609
XlsxWriter package, used for working with	608
XlsxWriter Python package	609, 610
used, for working with XLSX files	608

Z

ZIP files	619, 620
zipfile module	619

W

WeasyPrint	621
웹 서버 게이트웨이 인터페이스 (WSGI)	679
Whitenoise	679
Windows	
pyenv, 설치 중	3

X

XLSX 파일	608
데이터, 통합 문서에 쓰기	611
데이터, 워크시트에 쓰기	610

형식이 지정된 스프레드시트, 개발 스프레드시트, 형식 지정된 통합 문서	615–618611–614608
통합 문서, 생성	610
워크시트, 생성	610
워크시트들	609
XlsxWriter 패키지, 작업에 사용됨	608
XlsxWriter 파이썬 패키지	609, 610
XLSX 파일 작업에 사용됨	608

Z

ZIP 파일zipfile 모듈	619, 620619
------------------	-------------